RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23334 – Fundamentals of Data Science Lab

Laboratory Record Note Book

Name :	**Thamizhselvan.V**
Year / Branch / Section :	**2nd Year / CSE / G**
University Register No. :	**2116250701780**
College Roll No. :	**250701780**
Semester :	**Third Semester**
Academic Year :	**2026·2027**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 – Fundamentals of Data Science Lab

NAME	**Thamizhselvan.V**
ROLL NO	**250701780**
YEAR	**2nd Year**
SEC	**G**

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: **Thamizhselvan.V**

Academic Year: 2026-2027 Semester: Third Branch: CSE

Register No. **2116250701780**

Certified that this is the bonafide record of work done by the above student in the **Fundamentals of Data Science** ***Laboratory during the academic year 2026 – 2027.***

Signature of Faculty in-charge

Submitted for the Practical Examination held on ……

Internal Examiner External Examiner

INDEX

Sl.NO	Experiment	Date	GitHub
1.a	Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.	29/07/26	
1.b	Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.	29/07/26	
1.c	Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.	05/08/26	
1.d	Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.	05/08/26	
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	12/08/26	
3.a	Conduct an experiment to show data visualization using line plot	19/08/26	
3.b	Conduct an experiment to show data visualization using bar chart	19/08/26	
3.c	Conduct an experiment to show data visualization using pie chart	19/08/26	
4	To Count the frequency of occurrence of a word in a body of text is often needed during text processing.	26/08/26	
5	Data Collection and Initial Exploration	16/09/26	
6	Time Series Analysis of Website Traffic	16/09/26	
7	Random Sampling and Sampling	23/09/26	

	Distribution		
8	Conduct Z test for the Data given using Python	23/09/26	
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	23/09/26	
10	Experiment to understand array function in Data science.	23/09/26	
11	Experiment to understand pandas library use cases in Data science.	30/09/26	
12	Experiment to detect outliers in a given data set.	30/09/26	
13	Experiment to handle missing values and inappropriate data in a given data set.	30/09/26	
14	Experiment to understand feature scaling	30/09/26	
15	Experiment to understand the data preprocessing in Data science	30/09/26	
16	Experiment to understand EDA-Quantitative and Qualitative analysis.	07/10/26	
17	Experiment to understand Linear Regression for a given data set.	07/10/26	
18	Experiment to understand KNN algorithm for a given data set.	07/10/26	
19	Experiment to understand Logistic Regression for a given data set.	07/10/26	
20	Experiment to understand K-means clustering algorithm for a given data set.	07/10/26	

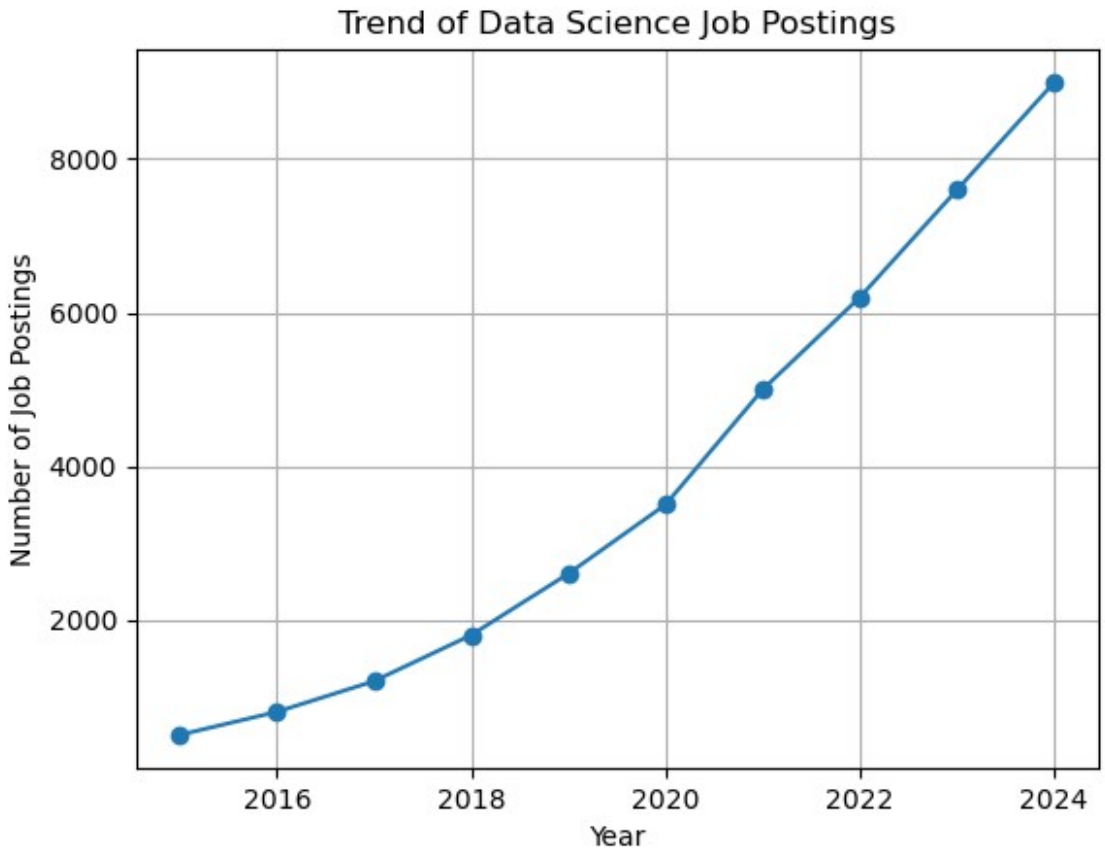
Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Exploratory Data Analysis to Develop Deep Insights from a Dataset

1.a. Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib for visualization.

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    "Year": [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023,
2024],
    "Job_Postings": [500, 800, 1200, 1800, 2600, 3500, 5000, 6200,
7600, 9000]
}
df=pd.DataFrame(data)
print(df)
plt.plot(df["Year"], df["Job_Postings"], marker='o')
plt.title("Trend of Data Science Job Postings")
plt.xlabel("Year")
plt.ylabel("Number of Job Postings")
plt.grid(True)
plt.show()
```

```
   Year  Job_Postings
0  2015           500
1  2016           800
2  2017          1200
3  2018          1800
4  2019          2600
5  2020          3500
6  2021          5000
7  2022          6200
8  2023          7600
9  2024          9000
```

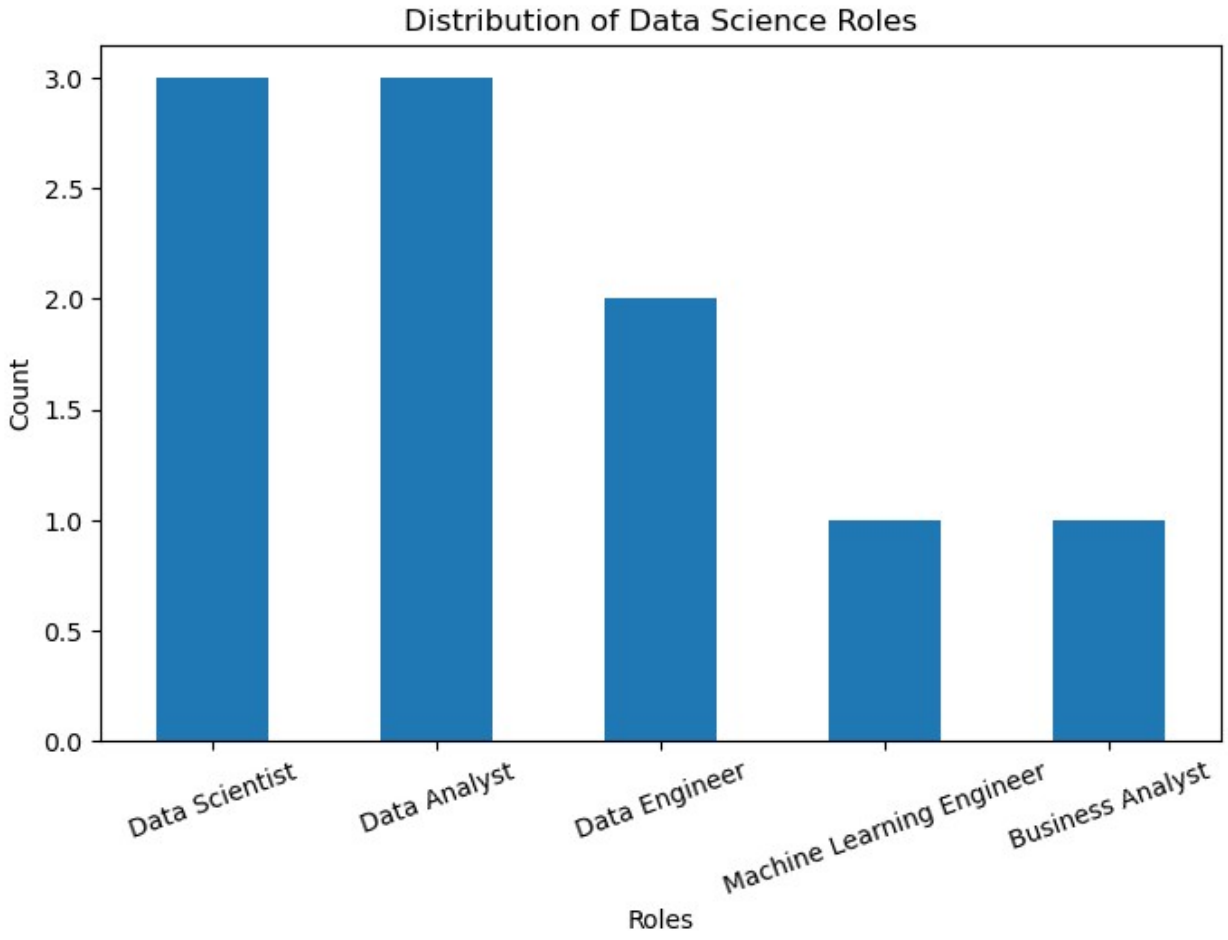


1.b.Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    "Role":[
        "Data Scientist",
        "Data Analyst",
        "Data Engineer",
        "Data Scientist",
        "Machine Learning Engineer",
        "Data Analyst",
        "Data Engineer",
        "Data Scientist",
        "Business Analyst",
        "Data Analyst"
    ]
}
df=pd.DataFrame(data)
df
role_count = df ["Role"].value_counts()
print(role_count)
```

```
plt.figure(figsize=(8,5))
role_count.plot(kind="bar")
plt.title("Distribution of Data Science Roles")
plt.xlabel("Roles")
plt.ylabel("Count")
plt.xticks(rotation=20)
plt.show()
```

```
Role
Data Scientist                3
Data Analyst                  3
Data Engineer                 2
Machine Learning Engineer     1
Business Analyst              1
Name: count, dtype: int64
```



1.c.Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

```
import pandas as pd
structured_data = {
    "ID": [101, 102, 103],
    "Name": ["Ravi", "Priya", "Arun"],
    "Age": [20, 21, 22]
}
df = pd.DataFrame (structured_data)
print("Structured Data:")
print(df)
print()
semi_structured_data = {
    "Student": {
        "ID": 101,
        "Name": "Ravi",
        "Department": "CSE"
    }
}
print("Semi-Structured Data:")
print(semi_structured_data)
print()
unstructured_data ="""
Artificial Intelligence is transforming industries.
Images, videos, audio files, and text documents are examples of
unstructured data.
"""
print("Unstructured Data:")
print(unstructured_data)
```

```
Structured Data:
    ID   Name  Age
0  101   Ravi   20
1  102  Priya   21
2  103   Arun   22

Semi-Structured Data:
{'Student': {'ID': 101, 'Name': 'Ravi', 'Department': 'CSE'}}

Unstructured Data:

Artificial Intelligence is transforming industries.
Images, videos, audio files, and text documents are examples of
unstructured data.
```

1.d.Conduct an experiment to encrypt and decrypt given sensitive data using the cryptography library fernet.

```
from cryptography.fernet import Fernet
key=Fernet.generate_key()
print("Secret Key:", key)
```

```
cipher=Fernet(key)
message = "Password123"
encrypted_message=cipher.encrypt(message.encode())
print("\nEncrypted Message:")
print(encrypted_message)
decrypted_message = cipher.decrypt(encrypted_message)
print("\nDecrypted Message:")
print(decrypted_message.decode())
```

```
Secret Key: b'RwIN0HBJ7mOFh0DD-lVVkW7Wcs3ykQbd0u6VCHr4Og0='

Encrypted Message:
b'gAAAAABqb1-
gvxq7IUyo5AZcTNVy6zt5aDaLUOdmjWBWFaxq6D7OJLC6qIcN8hdu5TAIIILfd62gbBjyl
wkhEpKxIpQQQJpPkA=='

Decrypted Message:
Password123
```

Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Upload and Analyze the Dataset in CSV Format and Perform Data Preprocessing and Visualization

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# ---------------------------------------------------
# 1. CREATE DATASET
#----------------------------------------------------
data = {
    "Date":[
        "01-01-2023", "02-01-2023", "03-01-2023", "04-01-2023",
        "05-01-2023", "06-01-2023", "07-01-2023", "08-01-2023",
        "09-01-2023", "10-01-2023", "11-01-2023", "12-01-2023",
        "13-01-2023", "14-01-2023", "15-01-2023", "16-01-2023"
    ],
    "Product": [
        "Product A", "Product B", "Product A", "Product C",
        "Product B", "Product A", "Product C", "Product B",
        "Product A", "Product C", "Product B", "Product A",
        "Product C", "Product B", "Product A", "Product C"
    ],
    "Sales": [
        200, 150, 220, 300, 180, 210, 320, 160,
        230, 310, 190, 240, 330, 170, 250, 340
    ],
    "Quantity": [
        4, 3, 5, 6, 4, 5, 7, 3,
        6, 7, 4, 6, 8, 3, 7, 8
    ],
    "Region": [
        "North", "South", "North", "East",
        "West", "North", "East", "South",
        "North", "East", "West", "North",
        "East", "South", "North", "East"
    ]
}
# Create DataFrame
df = pd.DataFrame(data)
# Convert Date column
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)
# ---------------------------------------------------
# 2. DISPLAY DATA
# ---------------------------------------------------
print("DATASET:")
print(df.head())
print("\n")
# ---------------------------------------------------
# 3. CHECK MISSING VALUES
```

```
# ------------------------------------------------
print("MISSING VALUES:")
print(df.isnull().sum())
print("\n")
# ------------------------------------------------
# 4. STATISTICAL SUMMARY
# ------------------------------------------------
print("STATISTICAL SUMMARY:")
print(df[["Sales", "Quantity"]].describe())
print("\n")
# ------------------------------------------------
# 5. PRODUCT-WISE SALES AND QUANTITY
# ------------------------------------------------
product_summary = df.groupby("Product")[["Sales",
"Quantity"]].sum().reset_index()
print("PRODUCT-WISE SALES AND QUANTITY:")
print(product_summary)
print("\n")
# ------------------------------------------------
# 6. LINE GRAPH - SALES OVER DATE
# ------------------------------------------------
plt.figure(figsize=(10, 5))
plt.plot(
    df["Date"],
    df["Sales"],
    marker="o",
    color="blue",
    linewidth=2
)
plt.title("Sales Over Time")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
# ------------------------------------------------
# 7. BAR GRAPH - PRODUCT-WISE SALES
# ------------------------------------------------
plt.figure(figsize=(8, 5))
plt.bar(
    product_summary["Product"],
    product_summary["Sales"],
    color=["blue", "orange", "green"]
)
plt.title("Product-wise Total Sales")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.tight_layout()
```

```python
plt.show()
# ------------------------------------------------
# 8. CORRELATION MATRIX
# ------------------------------------------------
correlation = df[["Sales", "Quantity"]].corr()
print("CORRELATION MATRIX:")
print(correlation)
print("\n")
# ------------------------------------------------
# 9. CORRELATION HEATMAP
# ------------------------------------------------
plt.figure(figsize=(7, 5))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    linewidths=0.5,
    fmt=".2f"
)
plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()
```

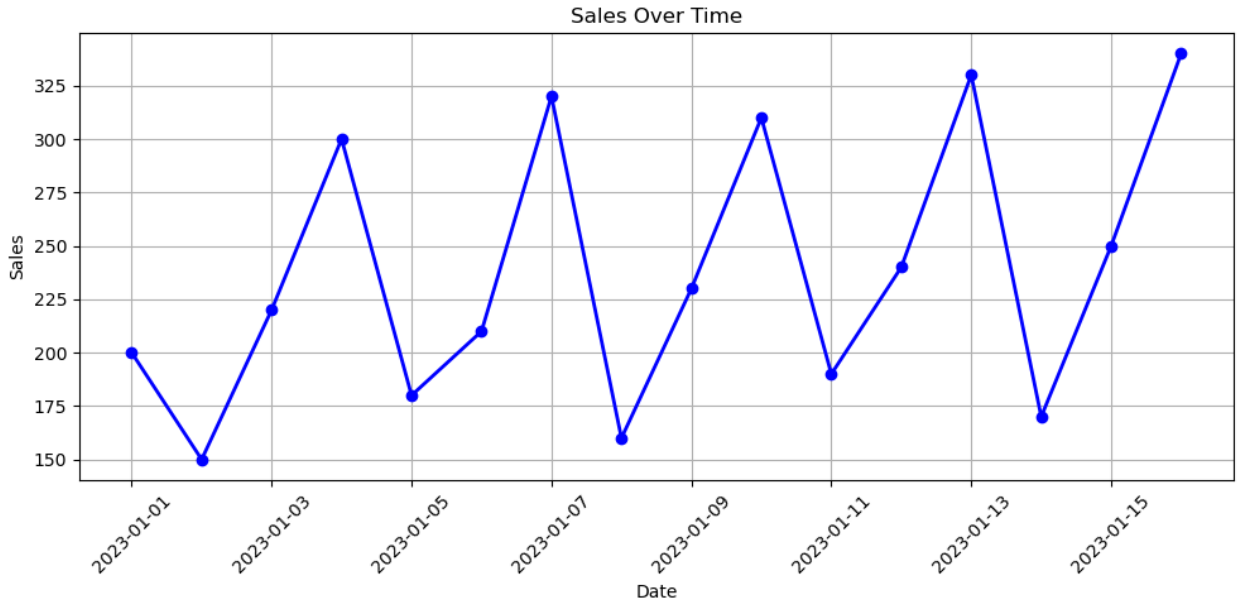
```
DATASET:
        Date    Product  Sales  Quantity Region
0 2023-01-01  Product A    200         4  North
1 2023-01-02  Product B    150         3  South
2 2023-01-03  Product A    220         5  North
3 2023-01-04  Product C    300         6   East
4 2023-01-05  Product B    180         4   West

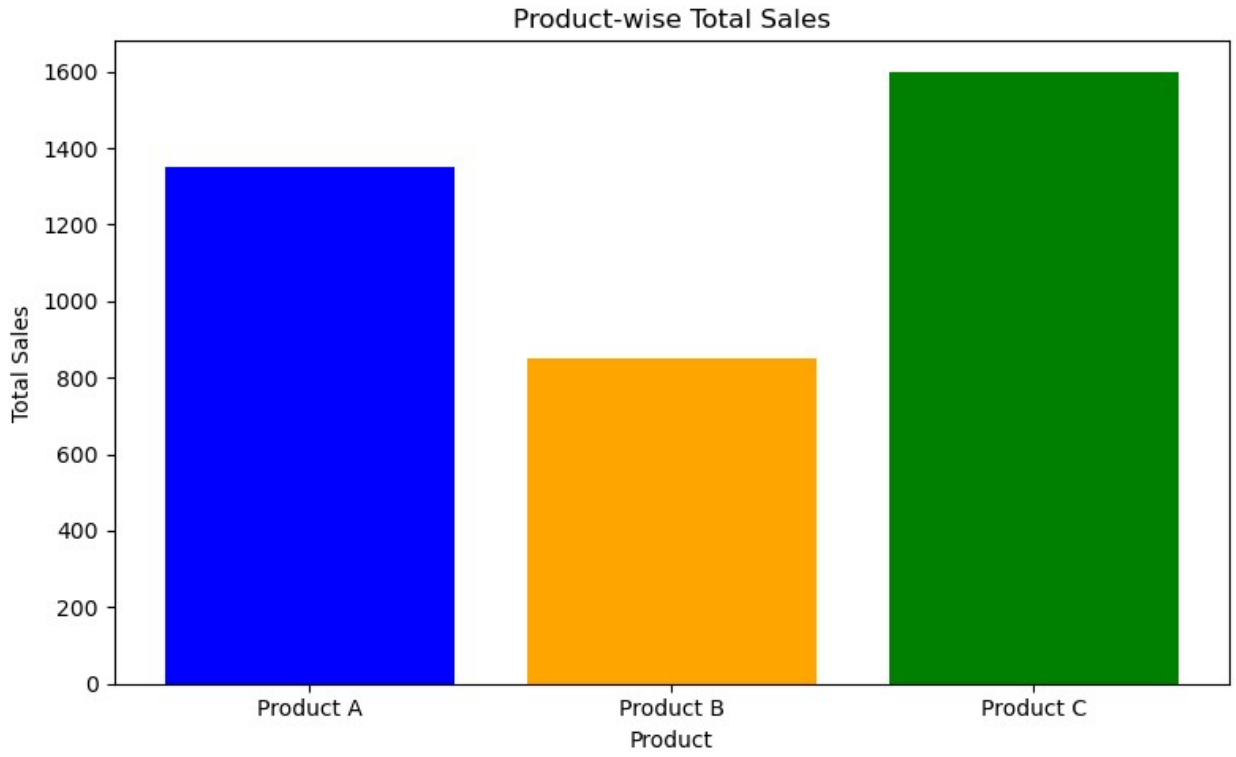

MISSING VALUES:
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64


STATISTICAL SUMMARY:
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
std     64.031242   1.746425
min    150.000000   3.000000
25%    187.500000   4.000000
50%    225.000000   5.500000
75%    302.500000   7.000000
```

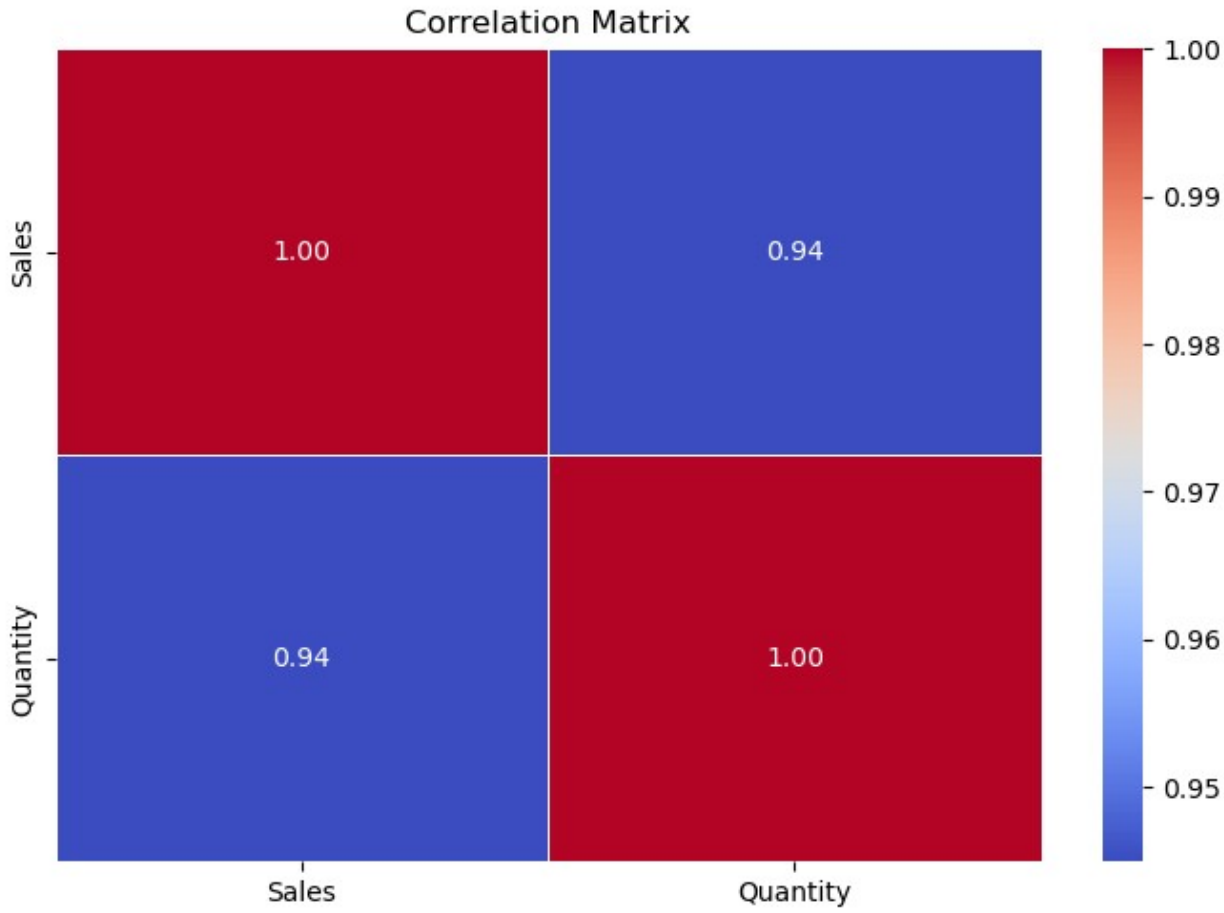
```
max    340.000000   8.000000


PRODUCT-WISE SALES AND QUANTITY:
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```






```
CORRELATION MATRIX:
              Sales  Quantity
Sales     1.000000  0.944922
Quantity  0.944922  1.000000
```


Correlation Matrix
Sales
Quantity
1.00
0.94
0.94
1.00
Sales
Quantity
1.00
0.99
0.98
0.97
0.96
0.95

Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Handling Missing and Inappropriate Data in a Dataset

```
import numpy as np
import pandas as pd
df = pd.read_csv("Hotel_Dataset.csv")
print("--- Initial Duplicate Identification ---")
print(df.duplicated())
print("\n--- Dataframe Info Summary ---")
df.info()
df.drop_duplicates(inplace=True)
index = np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
df.drop(["Age_Group.1"], axis=1, inplace=True)
df.CustomerID.loc[df.CustomerID < 0] = np.nan
df.Bill.loc[df.Bill < 0] = np.nan
df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
df["NoOfPax"].loc[(df["NoOfPax"] < 1) | (df["NoOfPax"] > 20)] = np.nan
print("\n--- Unique Categorical Value Sets ---")
print("Age Groups:", df.Age_Group.unique())
print("Hotels:", df.Hotel.unique())
df.Hotel.replace(["Ibys"], "Ibis", inplace=True)
df.FoodPreference.replace(["Vegetarian", "veg"], "Veg", inplace=True)
df.FoodPreference.replace(["non-Veg"], "Non-Veg", inplace=True)
df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()),
inplace=True)
df.NoOfPax.fillna(round(df.NoOfPax.median()), inplace=True)
df["Rating(1-5)"].fillna(round(df["Rating(1-5)"].median()),
inplace=True)
df.Bill.fillna(round(df.Bill.mean()), inplace=True)
print("\n--- Final Cleaned Dataset (df) ---")
print(df)
```

```
--- Initial Duplicate Identification ---
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool

--- Dataframe Info Summary ---
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     object
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     object
 4   FoodPreference   11 non-null     object
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     object
dtypes: int64(5), object(4)
memory usage: 924.0+ bytes

--- Unique Categorical Value Sets ---
Age Groups: ['20-25' '30-35' '25-30' '35+']
Hotels: ['Ibis' 'LemonTree' 'RedFox' 'Ibys']

--- Final Cleaned Dataset (df) ---
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill
\
0         1.0     20-25            4       Ibis            Veg  1300.0

1         2.0     30-35            5  LemonTree        Non-Veg  2000.0

2         3.0     25-30            6     RedFox            Veg  1322.0

3         4.0     20-25           -1  LemonTree            Veg  1234.0

4         5.0       35+            3       Ibis            Veg   989.0

5         6.0       35+            3       Ibis        Non-Veg  1909.0

6         7.0       35+            4     RedFox            Veg  1000.0

7         8.0     20-25            7  LemonTree            Veg  2999.0

8         9.0     25-30            2       Ibis        Non-Veg  3456.0

9        10.0     30-35            5     RedFox        Non-Veg  1801.0


   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
```

```
5      2.0         122220.0
6      2.0          21122.0
7      2.0         345673.0
8      3.0          96755.0
9      4.0          87777.0

C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:12:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0!
You are setting values through chained assignment. Currently this
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy.
A typical example is when you are setting values in a column of a
DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the
assignment in a single step and ensure this keeps updating the
original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy

  df.CustomerID.loc[df.CustomerID < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:12:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  df.CustomerID.loc[df.CustomerID < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:13:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0!
You are setting values through chained assignment. Currently this
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy.
A typical example is when you are setting values in a column of a
DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the
```

```
assignment in a single step and ensure this keeps updating the
original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy

  df.Bill.loc[df.Bill < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:13:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  df.Bill.loc[df.Bill < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:14:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0!
You are setting values through chained assignment. Currently this
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy.
A typical example is when you are setting values in a column of a
DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the
assignment in a single step and ensure this keeps updating the
original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy

  df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:14:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:15:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0!
You are setting values through chained assignment. Currently this
```

```
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy.
A typical example is when you are setting values in a column of a
DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the
assignment in a single step and ensure this keeps updating the
original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy

  df["NoOfPax"].loc[(df["NoOfPax"] < 1) | (df["NoOfPax"] > 20)] =
np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:15:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  df["NoOfPax"].loc[(df["NoOfPax"] < 1) | (df["NoOfPax"] > 20)] =
np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:19:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df.Hotel.replace(["Ibys"], "Ibis", inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:20:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
```

```
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df.FoodPreference.replace(["Vegetarian", "veg"], "Veg",
inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:24:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df["Rating(1-5)"].fillna(round(df["Rating(1-5)"].median()),
inplace=True)
```

Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Experiment to Understand Data Preprocessing in Data Science

```
import numpy as np
import pandas as pd

df = pd.read_csv('pre-process_datasample.csv')

df['Country'].fillna(df['Country'].mode(), inplace=True)
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Salary'].fillna(round(df['Salary'].mean()), inplace=True)

dummy_countries = pd.get_dummies(df['Country'])
updated_dataset = pd.concat([dummy_countries, df.iloc[:, 1:4]],
axis=1)
updated_dataset['Purchased'].replace(['No', 'Yes'], inplace=True)

print(updated_dataset)
```

```
   France  Germany  Spain   Age   Salary Purchased
0    True    False  False  44.0  72000.0        No
1   False    False   True  27.0  48000.0        No
2   False     True  False  30.0  54000.0        No
3   False    False   True  38.0  61000.0        No
4   False     True  False  40.0  63778.0        No
5    True    False  False  35.0  58000.0        No
6   False    False   True  38.0  52000.0        No
7    True    False  False  48.0  79000.0        No
8   False     True  False  50.0  83000.0        No
9    True    False  False  37.0  67000.0        No
```

```
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:6:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['Country'].fillna(df['Country'].mode(), inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:7:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
```

```
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['Age'].fillna(df['Age'].median(), inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:8:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['Salary'].fillna(round(df['Salary'].mean()), inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:12:
FutureWarning: Series.replace without 'value' and with non-dict-like
'to_replace' is deprecated and will raise in a future version.
Explicitly specify the new values instead.
  updated_dataset['Purchased'].replace(['No', 'Yes'], inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:12:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  updated_dataset['Purchased'].replace(['No', 'Yes'], inplace=True)
```

Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Outliers Detection

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

array = np.array([27, 50, 44, 6, 58, 61, 23, 86, 67, 20, 75, 7, 79,
61, 90, 54])

def outDetection(arr):
    Q1, Q3 = np.percentile(arr, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)
    return lr, ur

lr, ur = outDetection(array)

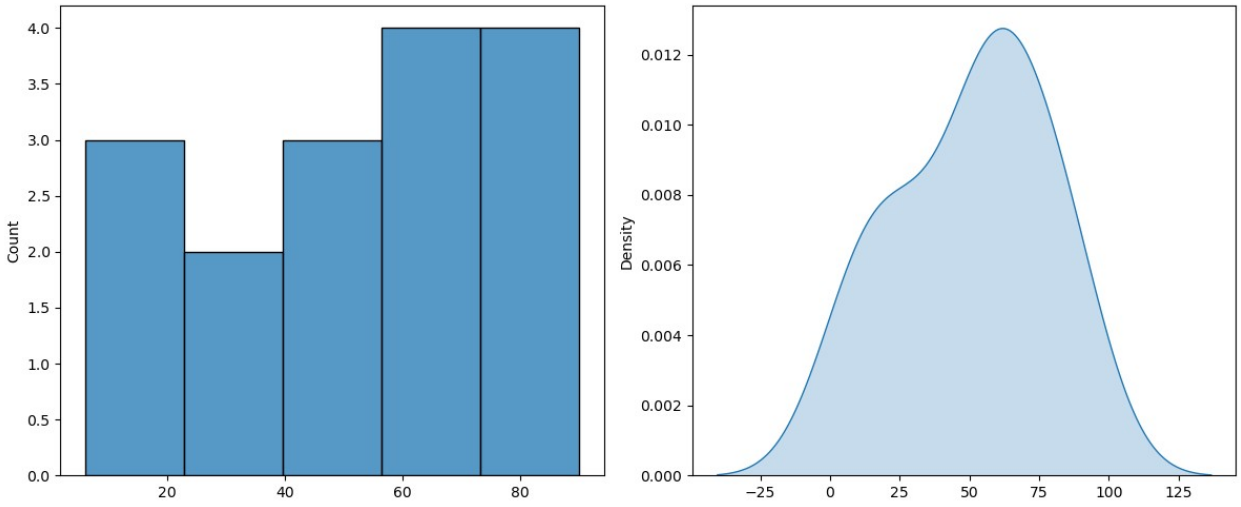
final_array = array[(array >= lr) & (array <= ur)]

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.histplot(final_array, kde=False)

plt.subplot(1, 2, 2)
sns.kdeplot(final_array, fill=True)

plt.tight_layout()
plt.show()
```



Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Feature Scaling

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler,
MinMaxScaler

df = pd.read_csv('pre-process_datasample.csv')

df['Country'] = df['Country'].fillna(df['Country'].mode()[0])

features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values

age = SimpleImputer(strategy="mean", missing_values=np.nan)
salary = SimpleImputer(strategy="mean", missing_values=np.nan)

features[:, 1:2] = age.fit_transform(features[:, 1:2])
features[:, 2:3] = salary.fit_transform(features[:, 2:3])

oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, 0:1])

final_set = np.concatenate((Country, features[:, 1:3]), axis=1)

sc = StandardScaler()
feat_standard_scaler = sc.fit_transform(final_set)
print(feat_standard_scaler)

mms = MinMaxScaler(feature_range=(0, 1))
feat_minmax_scaler = mms.fit_transform(final_set)
print(feat_minmax_scaler)
```

```
[[ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01  7.58874362e-01
   7.49473254e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00 -1.71150388e+00
  -1.43817841e+00]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01 -1.27555478e+00
  -8.91265492e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00 -1.13023841e-01
  -2.53200424e-01]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01  1.77608893e-01
   6.63219199e-16]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01 -5.48972942e-01
  -5.26656882e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00  0.00000000e+00
  -1.07356980e+00]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01  1.34013983e+00
   1.38753832e+00]
```

```
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01  1.63077256e+00
   1.75214693e+00]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01 -2.58340208e-01
   2.93712492e-01]]
[[1.         0.         0.         0.73913043 0.68571429]
 [0.         0.         1.         0.         0.        ]
 [0.         1.         0.         0.13043478 0.17142857]
 [0.         0.         1.         0.47826087 0.37142857]
 [0.         1.         0.         0.56521739 0.45079365]
 [1.         0.         0.         0.34782609 0.28571429]
 [0.         0.         1.         0.51207729 0.11428571]
 [1.         0.         0.         0.91304348 0.88571429]
 [0.         1.         0.         1.         1.        ]
 [1.         0.         0.         0.43478261 0.54285714]]
```

Name: Thamizhselvan.V Register Number: 250701780

Experiment Name: EDA - Quantitative and Qualitative Analysis

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

tips = sns.load_dataset('tips')

print(tips.head())

sns.displot(tips['total_bill'], kde=True)
plt.show()

sns.displot(tips['total_bill'], kde=False)
plt.show()

sns.jointplot(x='tip', y='total_bill', data=tips)
plt.show()

sns.jointplot(x='tip', y='total_bill', data=tips, kind='reg')
plt.show()

sns.jointplot(x='tip', y='total_bill', data=tips, kind='hex')
plt.show()

sns.pairplot(tips)
plt.show()

print(tips['time'].value_counts())

sns.pairplot(tips, hue='time')
plt.show()

sns.pairplot(tips, hue='day')
plt.show()

sns.heatmap(tips.corr(numeric_only=True), annot=True)
plt.show()

sns.boxplot(y=tips['total_bill'])
plt.show()

sns.boxplot(y=tips['tip'])
plt.show()

sns.countplot(y=tips['day'])
plt.show()
```

```
sns.countplot(y=tips['sex'])
plt.show()

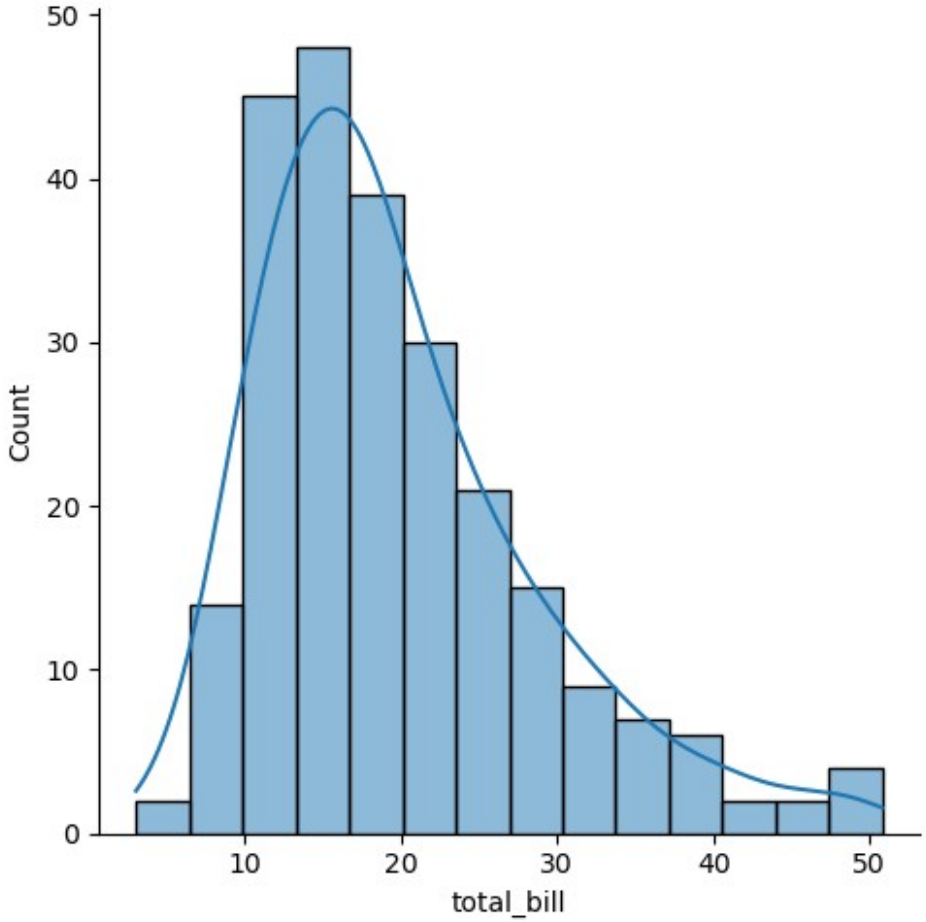
tips['sex'].value_counts().plot(kind='pie')
plt.show()

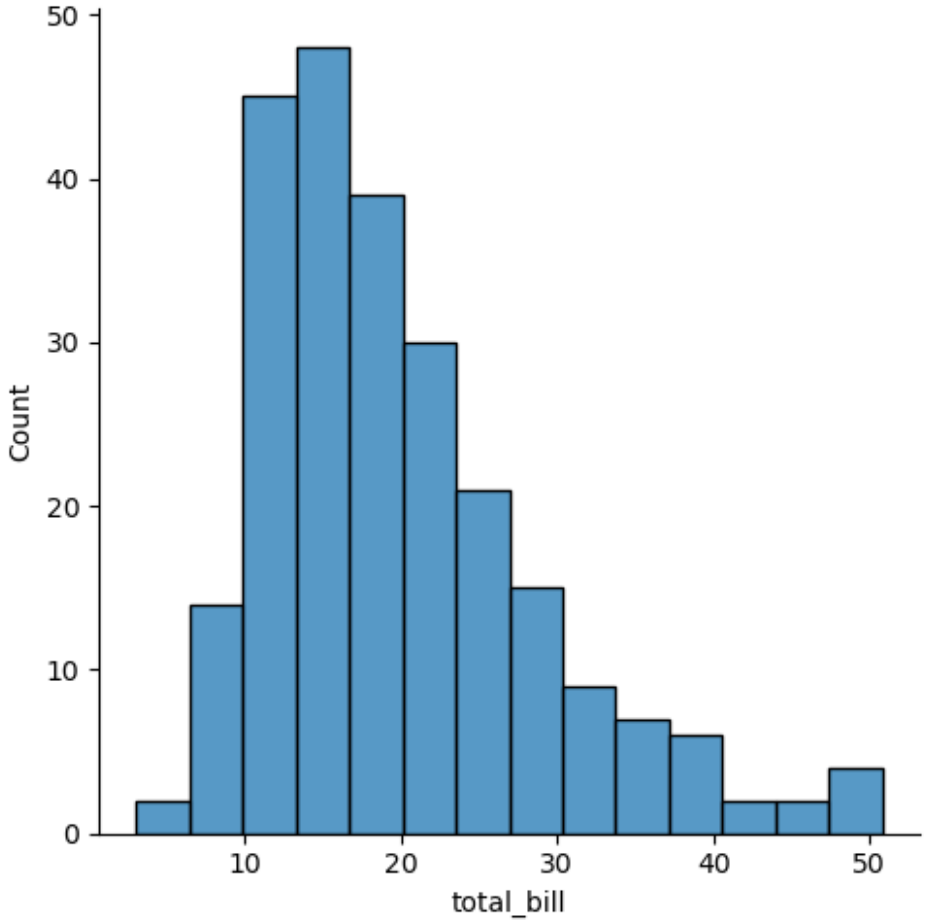
tips['sex'].value_counts().plot(kind='bar')
plt.show()

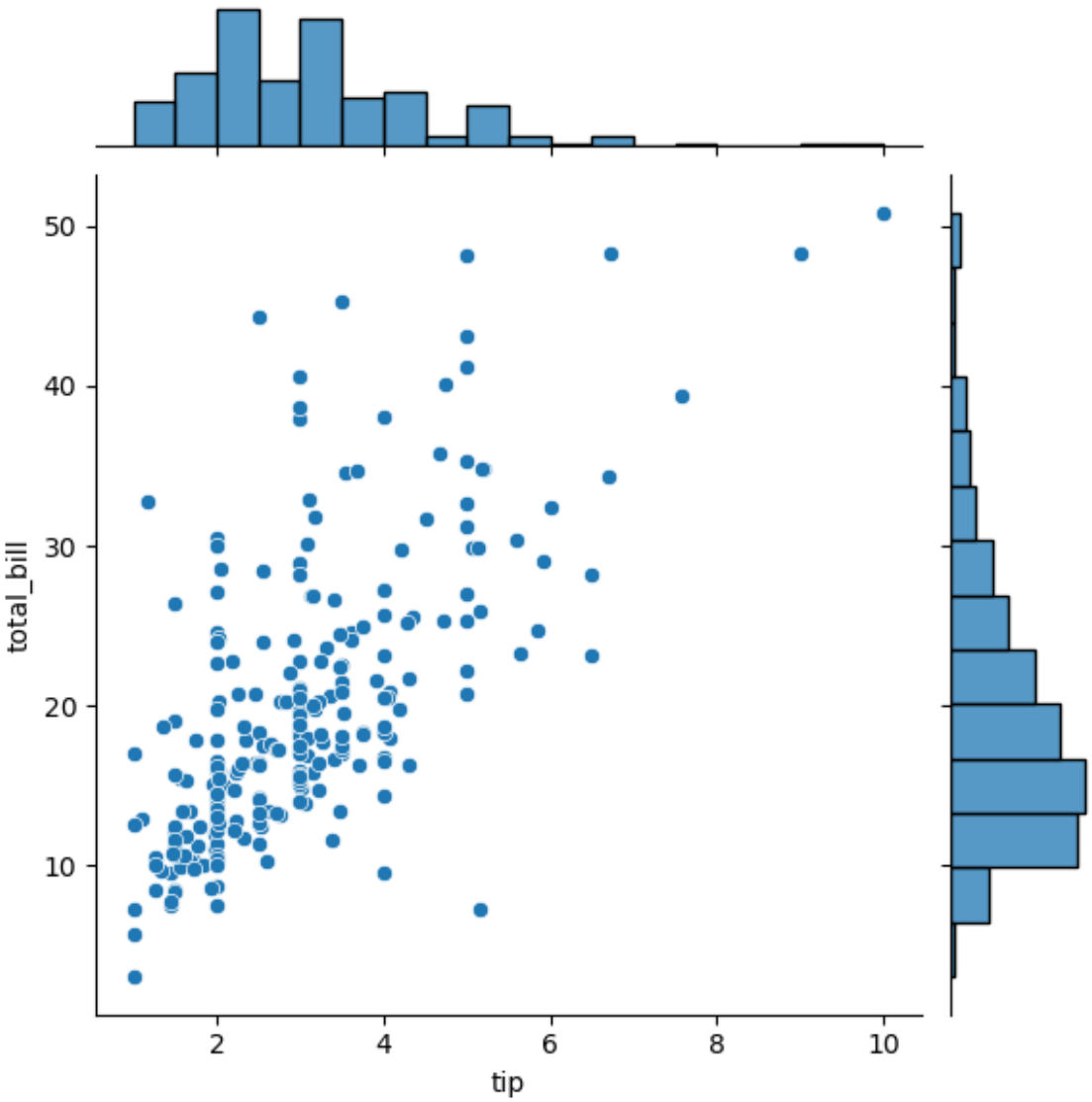
sns.countplot(x=tips['sex'])
plt.show()

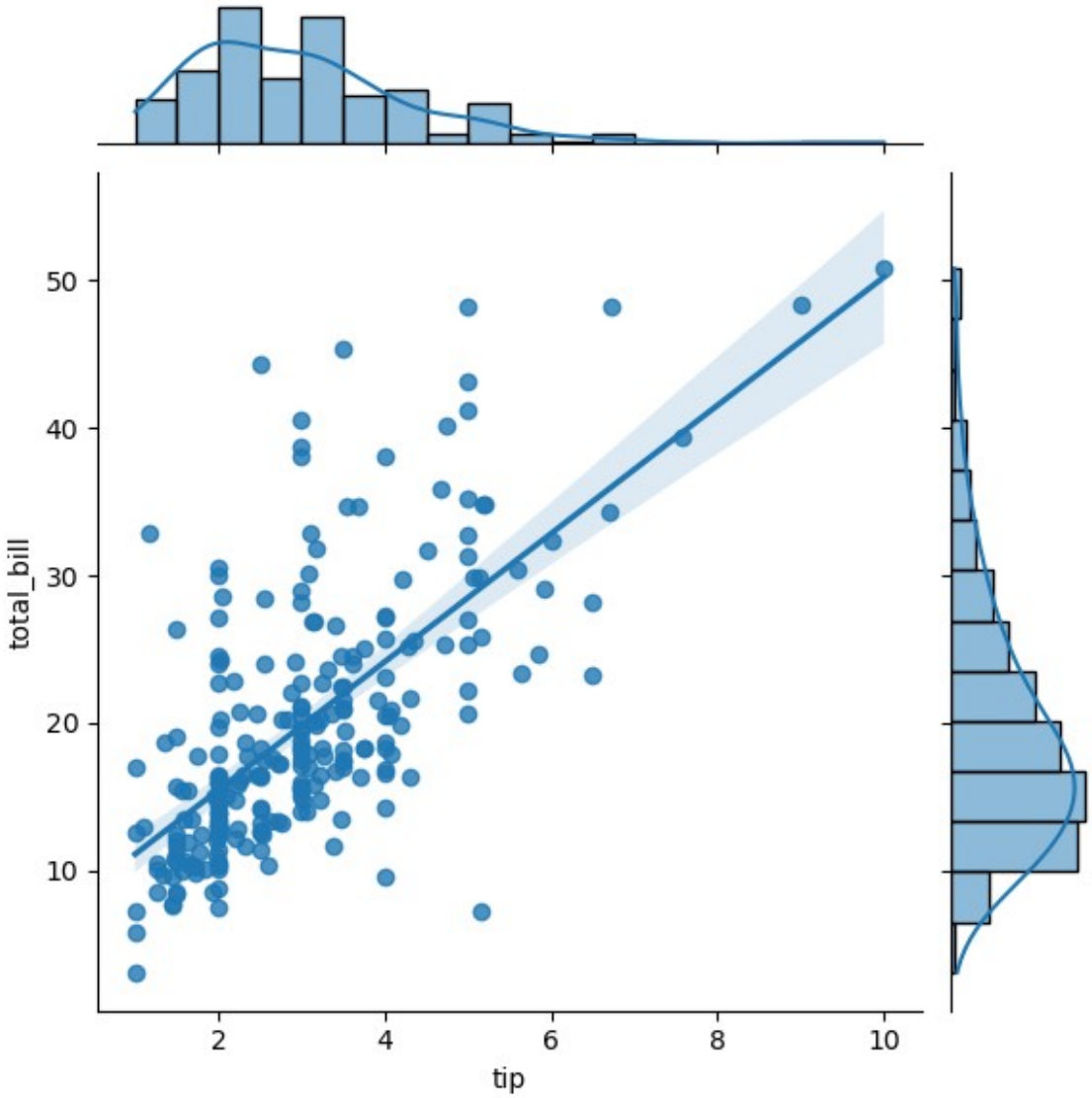
sns.countplot(data=tips[tips['time'] == 'Dinner'], x='day')
plt.show()
```

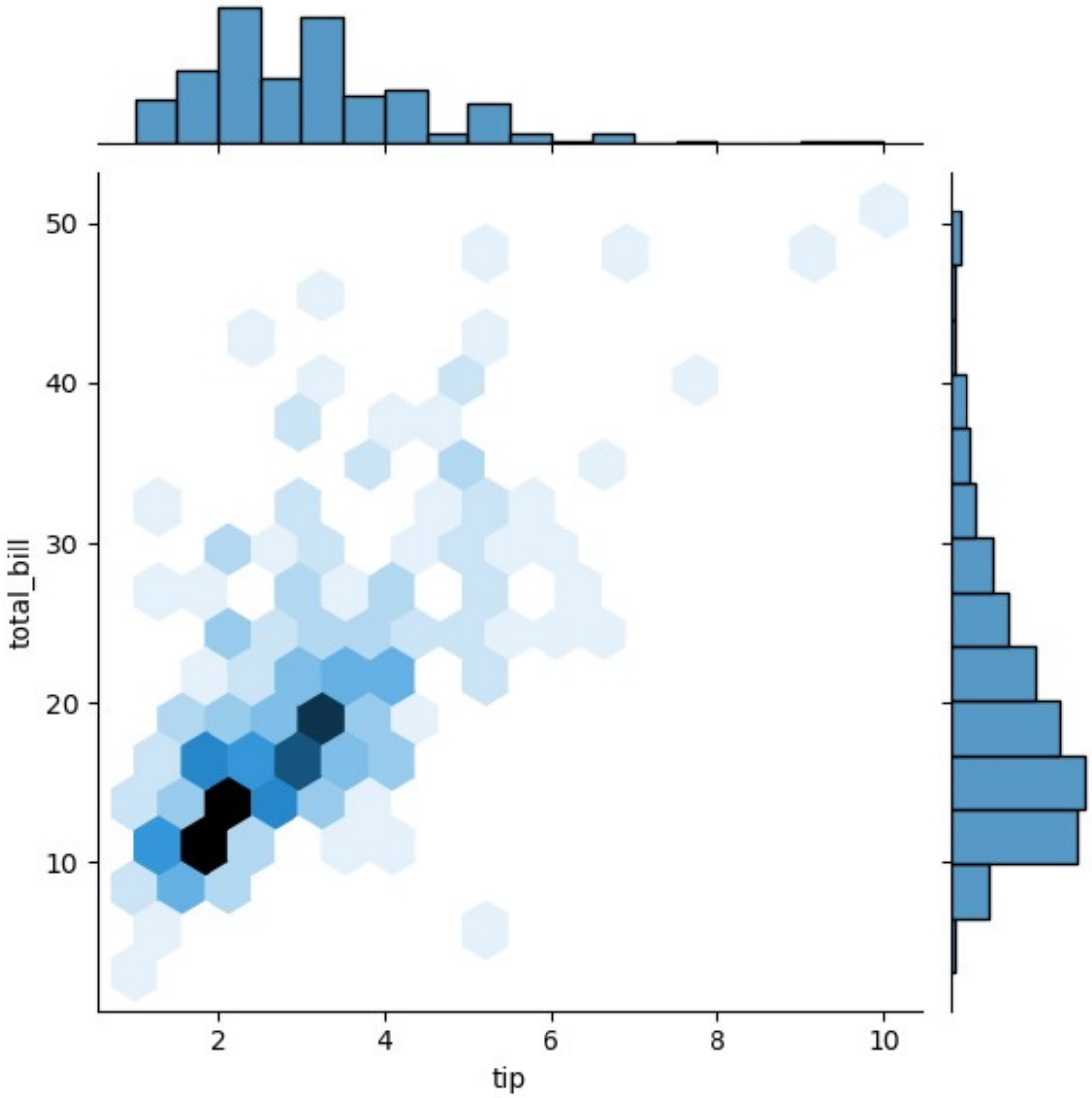
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

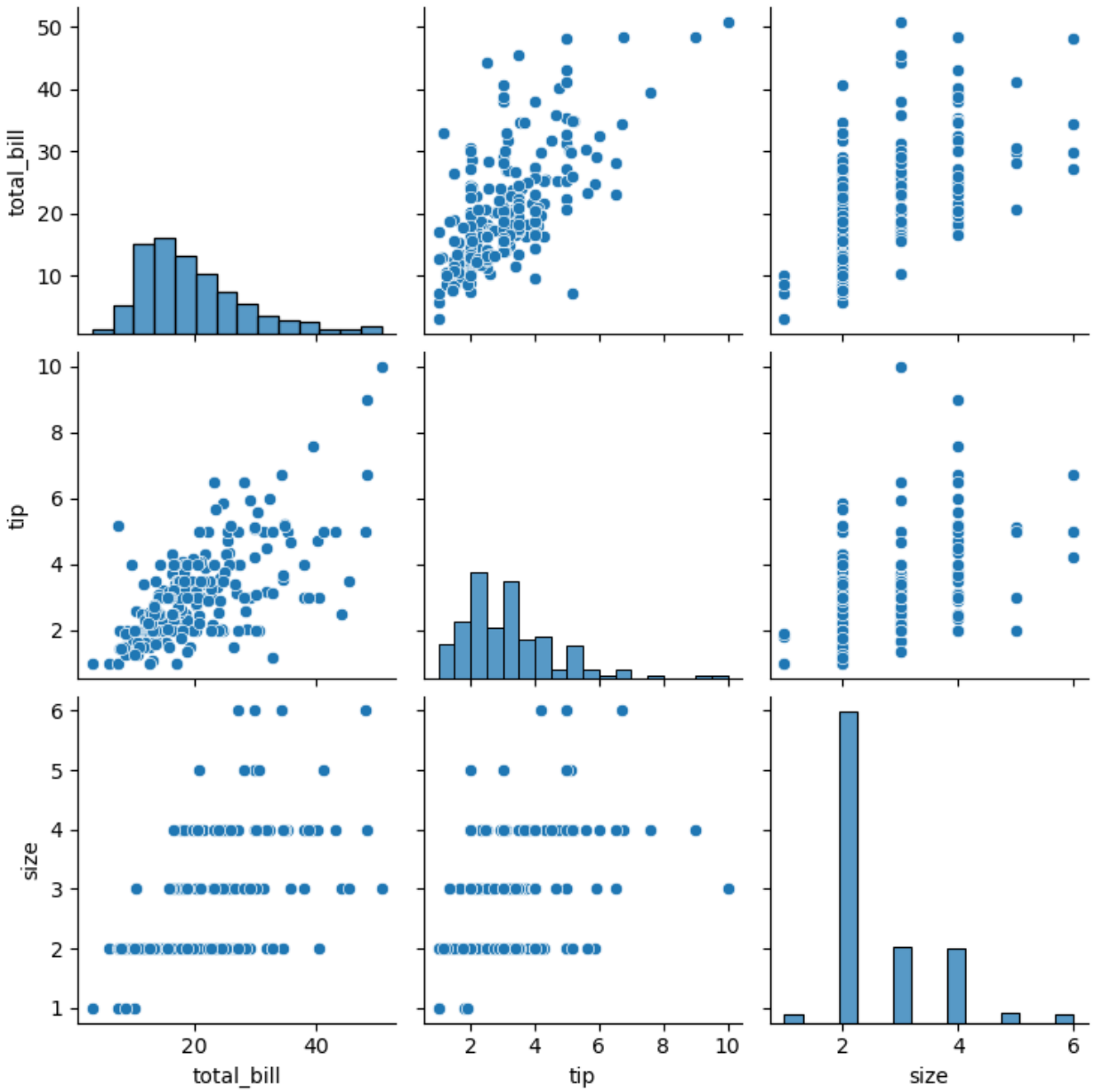

50
40
30
20
10
0
Count
10
20
30
40
50
total_bill


50
40
30
20
10
0
Count
10
20
30
40
50
total_bill

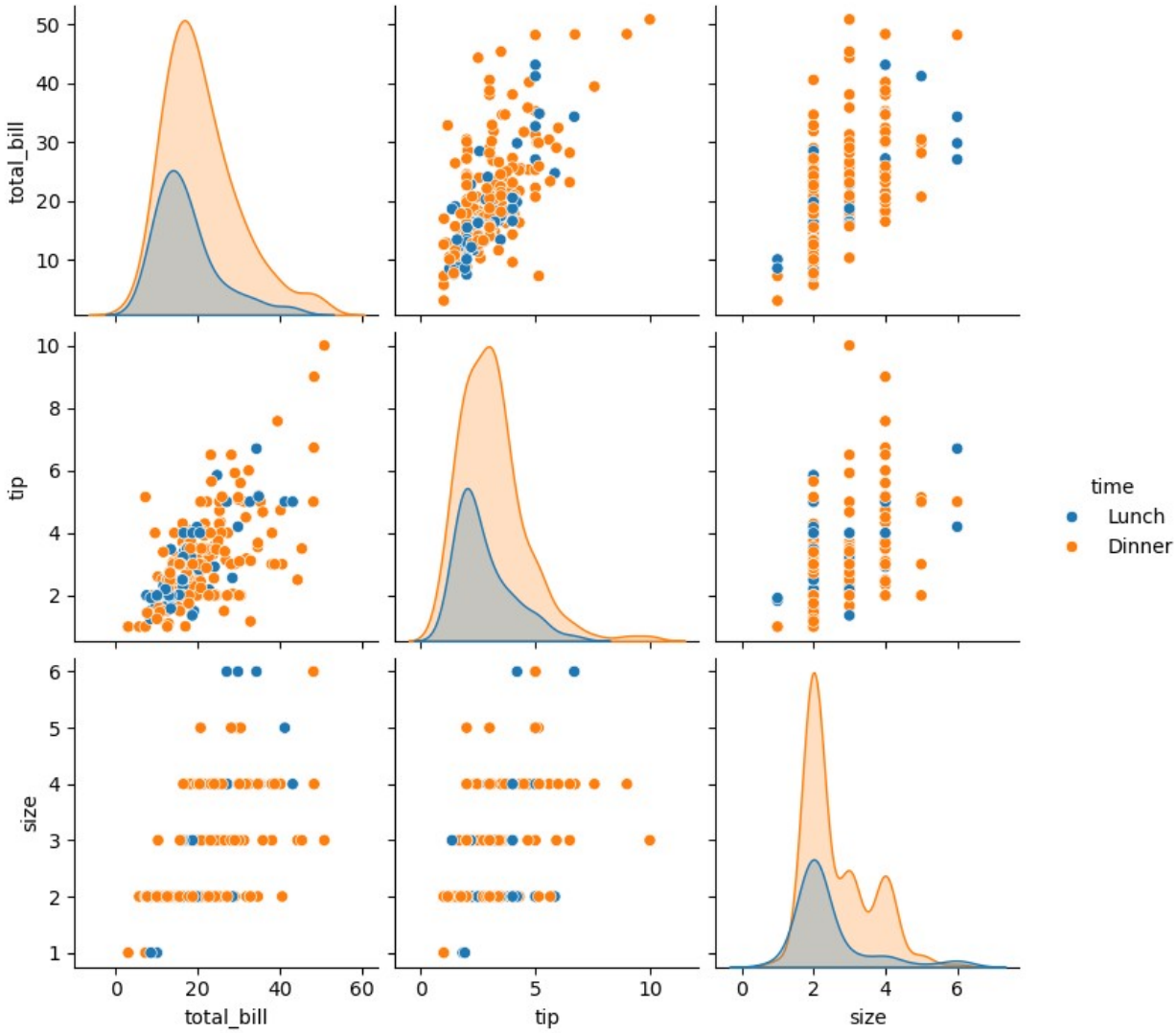

total_bill
tip
10
20
30
40
50
2
4
6
8
10

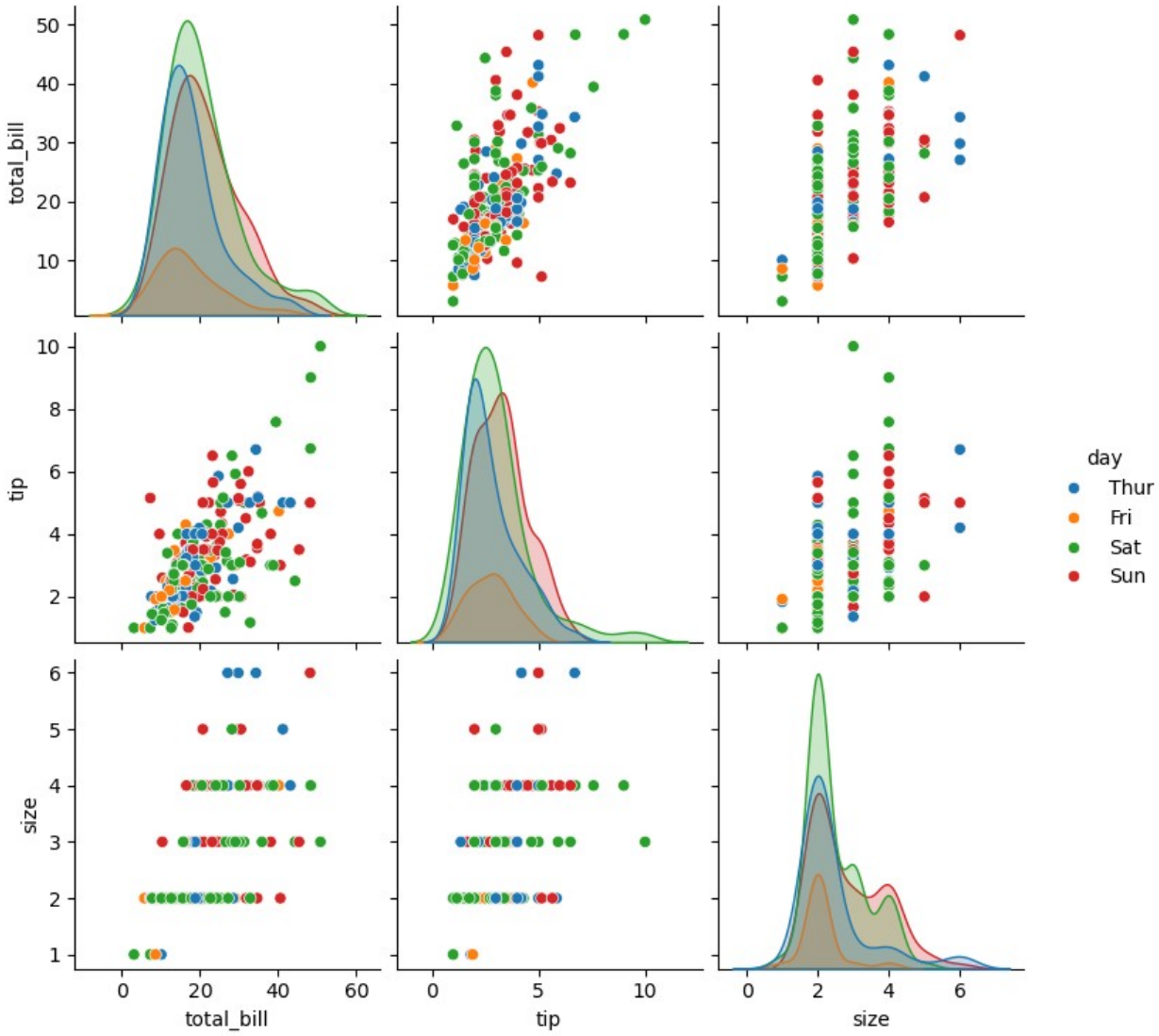

total_bill
tip
10
20
30
40
50
2
4
6
8
10

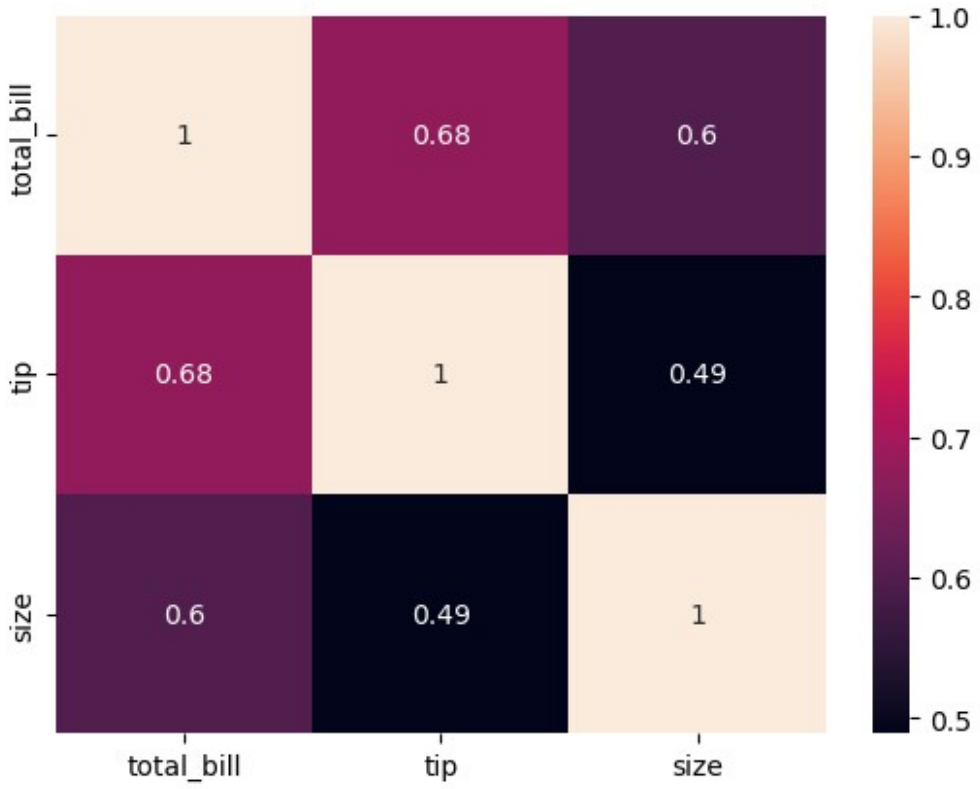

50
40
30
20
10
total_bill
2
4
6
8
10
tip

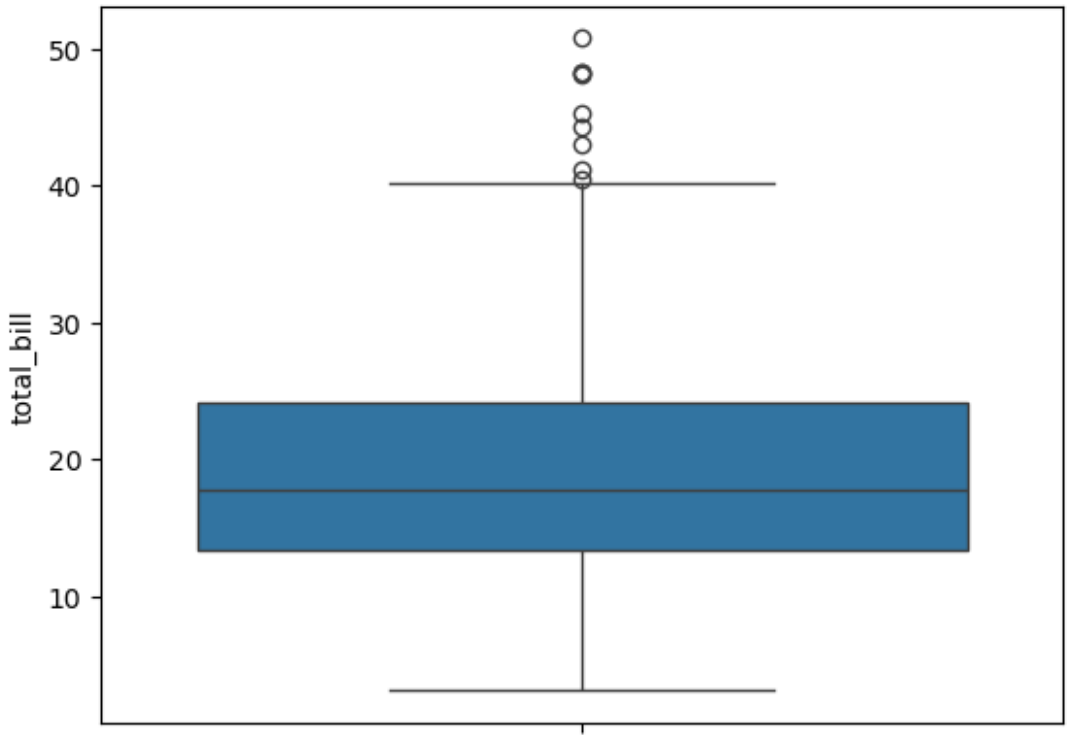


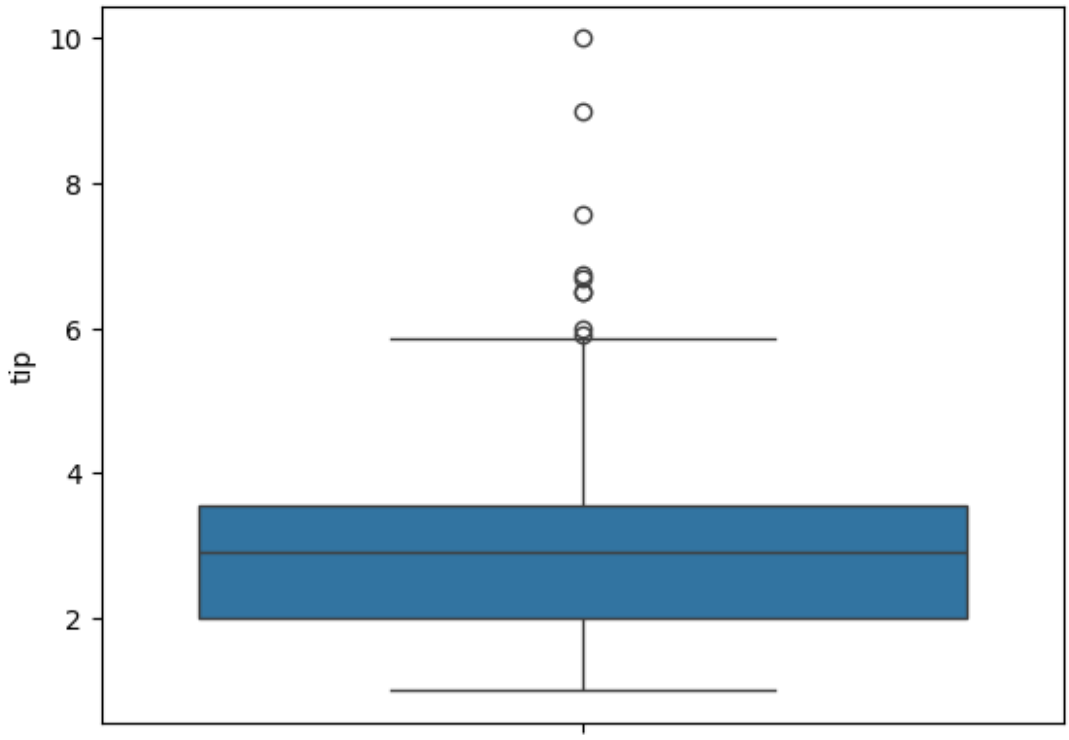

```
time
Dinner    176
Lunch      68
Name: count, dtype: int64
```

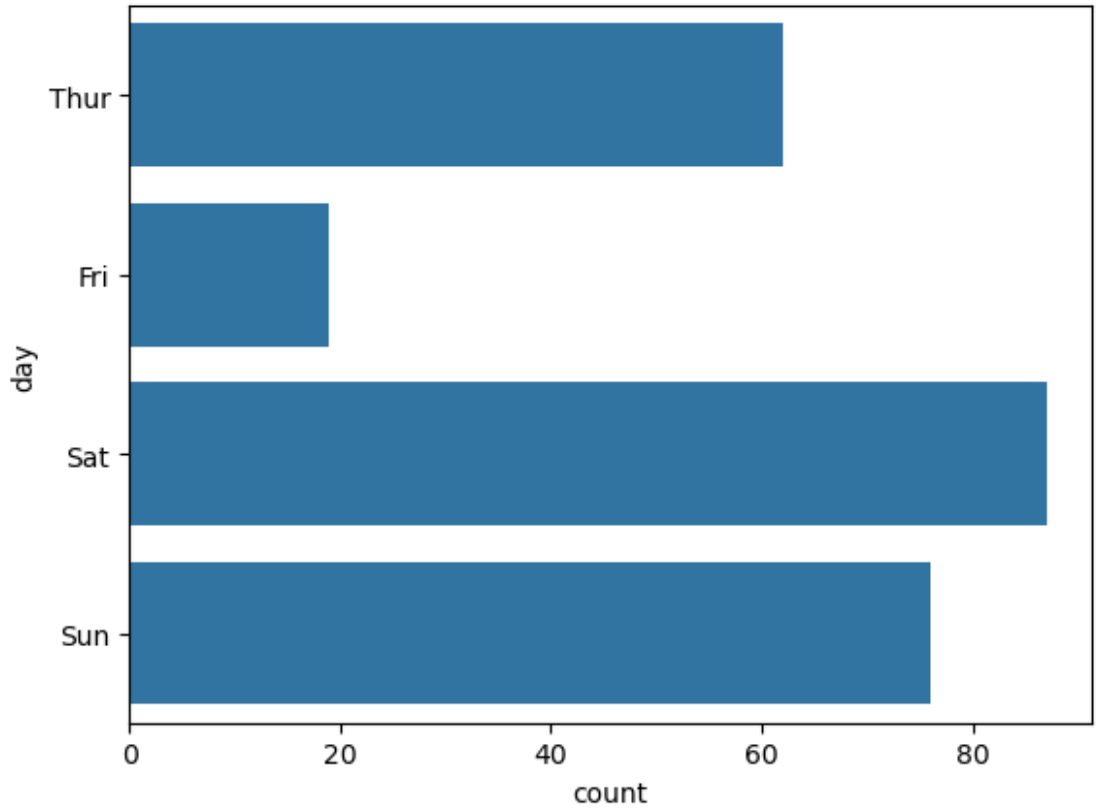

total_bill
tip
size
time
Lunch
Dinner

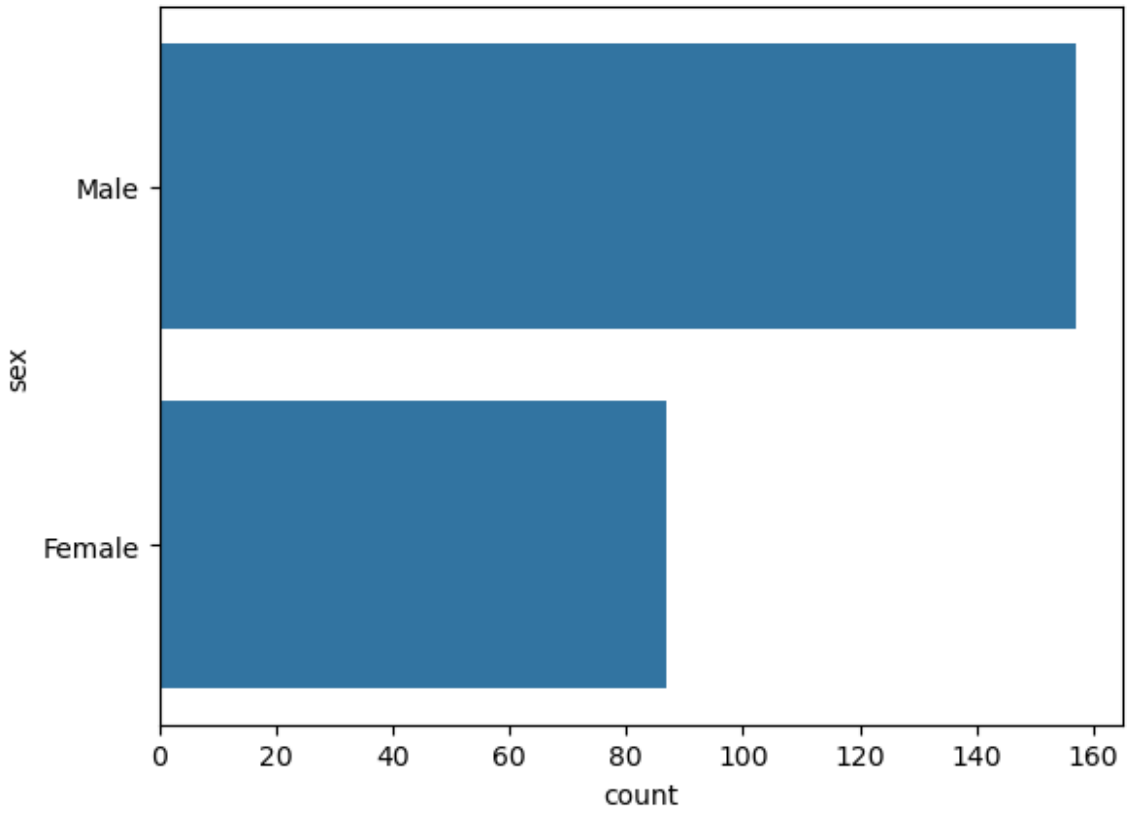

total_bill
tip
size
day
Thur
Fri
Sat
Sun

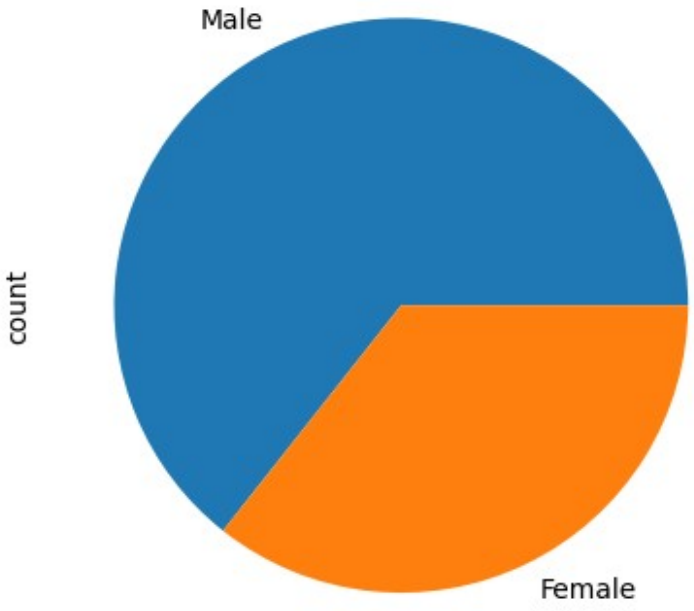

total_bill
tip
size
1
0.68
0.6
0.68
1
0.49
0.6
0.49
1
total_bill
tip
size
1.0
0.9
0.8
0.7
0.6
0.5

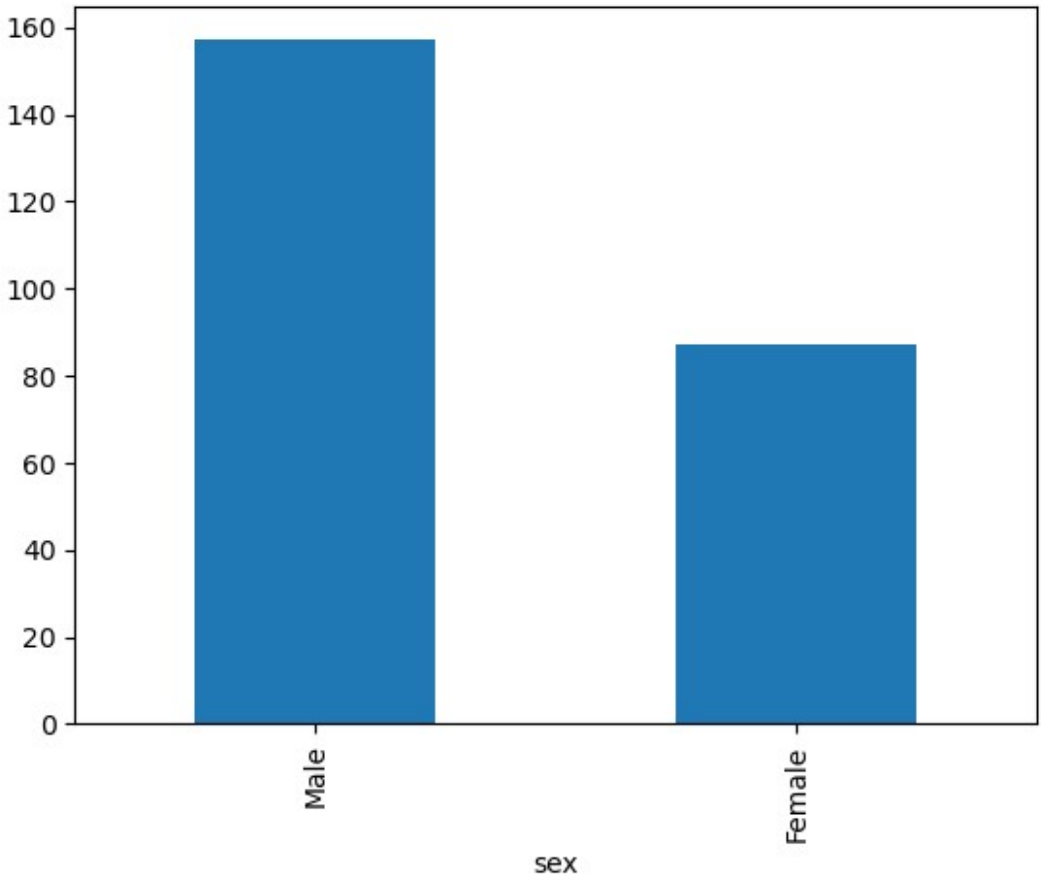

total_bill
50
40
30
20
10

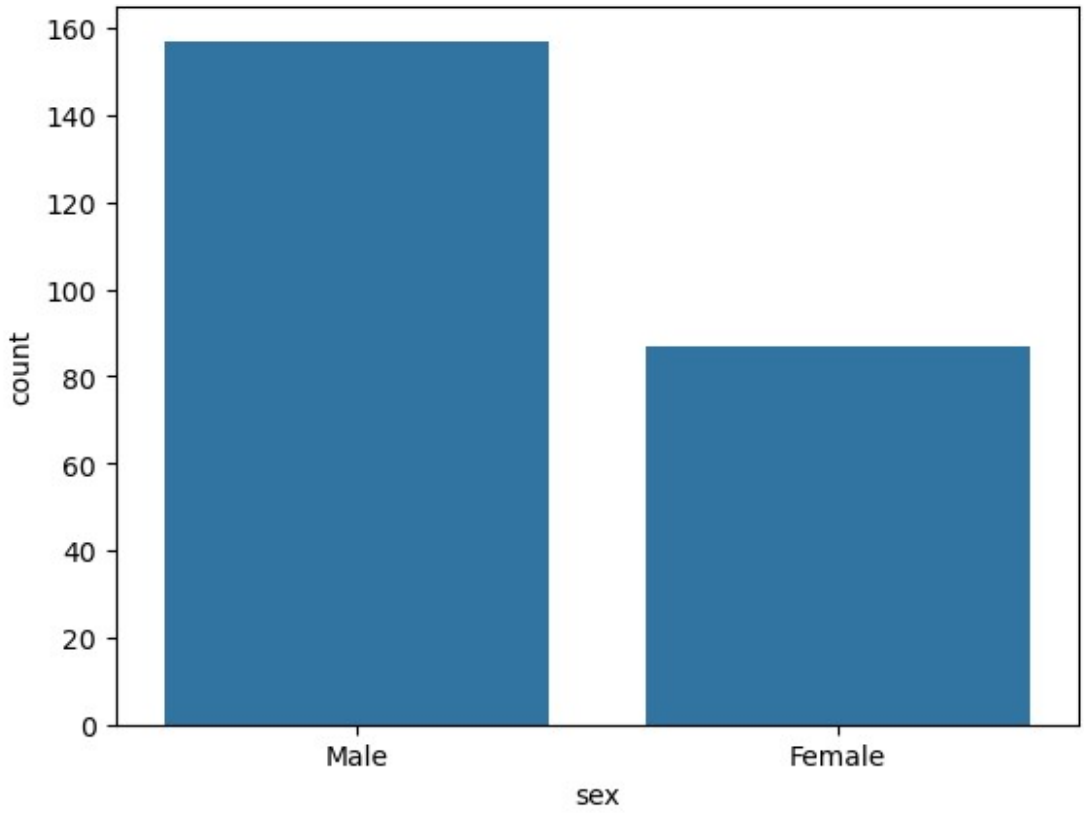

10
8
6
4
2
tip

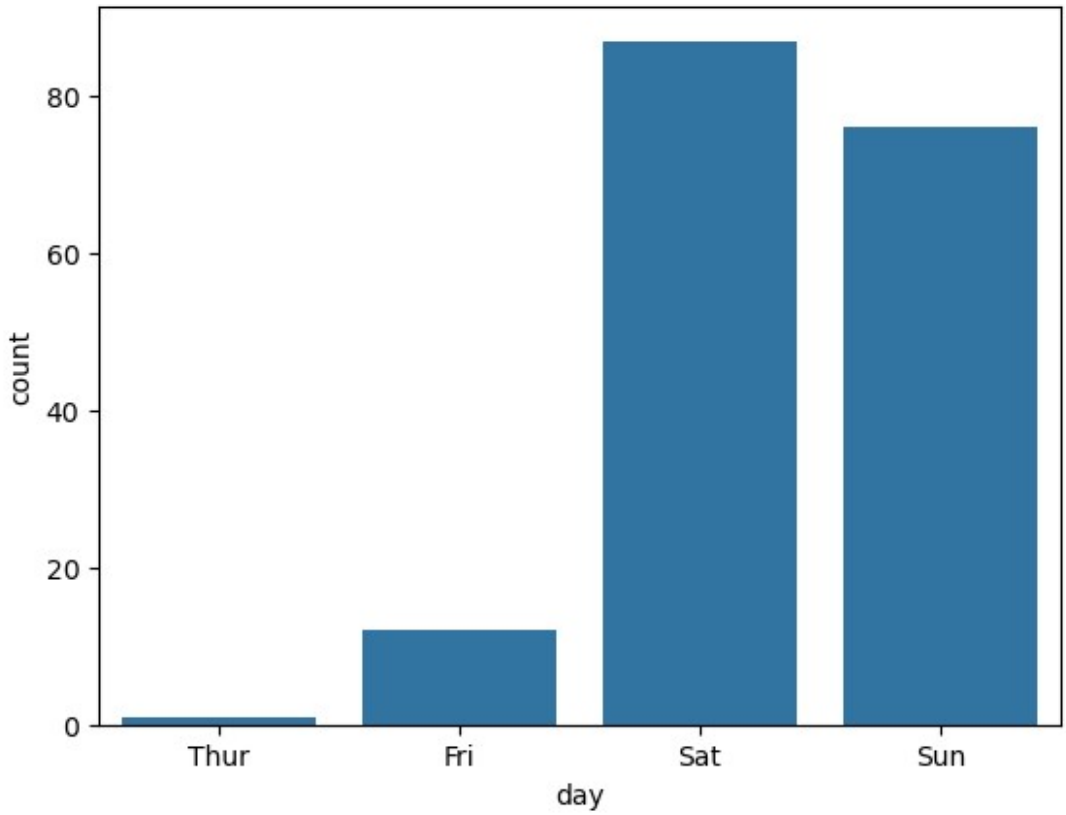

Thur
Fri
Sat
Sun
day
0
20
40
60
80
count


Male
Female
sex
0
20
40
60
80
100
120
140
160
count


Male
Female
count


160
140
120
100
80
60
40
20
0
Male
Female
sex


160
140
120
100
80
60
40
20
0
count
Male
Female
sex


80
60
40
20
0
count
Thur
Fri
Sat
Sun
day

Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Linear Regression

```
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data.csv')
df
```

```
    YearsExperience  Salary
0               1.1   39343
1               1.3   46205
2               1.5   37731
3               2.0   43525
4               2.2   39891
5               2.9   56642
6               3.0   60150
7               3.2   54445
8               3.2   64445
9               3.7   57189
10              3.9   63218
11              4.0   55794
12              4.0   56957
13              4.1   57081
14              4.5   61111
15              4.9   67938
16              5.1   66029
17              5.3   83088
18              5.9   81363
19              6.0   93940
20              6.8   91738
21              7.1   98273
22              7.9  101302
23              8.2  113812
24              8.7  109431
25              9.0  105582
26              9.5  116969
27              9.6  112635
28             10.3  122391
29             10.5  121872
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

```
df.dropna(inplace=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

```
df.describe()
```

```
       YearsExperience         Salary
count        30.000000      30.000000
mean          5.313333   76003.000000
std           2.837888   27414.429785
min           1.100000   37731.000000
25%           3.200000   56720.750000
50%           4.700000   65237.000000
75%           7.700000  100544.750000
max          10.500000  122391.000000
```

```
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,test_siz
e=0.2,random_state=42)
```

```
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
```

```
LinearRegression()
```

```
model.score(x_train,y_train)
```

```
0.9645401573418146
```

```
model.score(x_test,y_test)
```

```
0.9024461774180497
```

```
model.coef_
```

```
array([[9423.81532303]])
```

```
model.intercept_
```

```
array([25321.58301178])
```

```
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))

model=pickle.load(open('SalaryPred.model','rb'))

yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
```

```
Enter Years of Experience:  44
```

```
print("Estimated Salary for {} years of experience is {}:
" .format(yr_of_exp,Salary))
```

```
Estimated Salary for 44.0 years of experience is [[439969.45722514]]:
```

Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Logistic Regression

```
import numpy as np
import pandas as pd
df=pd.read_csv('Social_Network_Ads.csv')
df
```

```
      User ID  Gender  Age  EstimatedSalary  Purchased
0    15624510    Male   19            19000          0
1    15810944    Male   35            20000          0
2    15668575  Female   26            43000          0
3    15603246  Female   27            57000          0
4    15804002    Male   19            76000          0
..        ...     ...  ...              ...        ...
395  15691863  Female   46            41000          1
396  15706071    Male   51            23000          1
397  15654296  Female   50            20000          1
398  15755018    Male   36            33000          0
399  15594041  Female   49            36000          1

[400 rows x 5 columns]
```

```
df.head()
```

```
    User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19            19000          0
1  15810944    Male   35            20000          0
2  15668575  Female   26            43000          0
3  15603246  Female   27            57000          0
4  15804002    Male   19            76000          0
```

```
features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
features
```

```
array([[    19,  19000],
       [    35,  20000],
       [    26,  43000],
       [    27,  57000],
       [    19,  76000],
       [    27,  58000],
       [    27,  84000],
       [    32, 150000],
       [    25,  33000],
       [    35,  65000],
       [    26,  80000],
       [    26,  52000],
       [    20,  86000],
       [    32,  18000],
       [    18,  82000],
```

```
       [    29,  80000],
       [    47,  25000],
       [    45,  26000],
       [    46,  28000],
       [    48,  29000],
       [    45,  22000],
       [    47,  49000],
       [    48,  41000],
       [    45,  22000],
       [    46,  23000],
       [    47,  20000],
       [    49,  28000],
       [    47,  30000],
       [    29,  43000],
       [    31,  18000],
       [    31,  74000],
       [    27, 137000],
       [    21,  16000],
       [    28,  44000],
       [    27,  90000],
       [    35,  27000],
       [    33,  28000],
       [    30,  49000],
       [    26,  72000],
       [    27,  31000],
       [    27,  17000],
       [    33,  51000],
       [    35, 108000],
       [    30,  15000],
       [    28,  84000],
       [    23,  20000],
       [    25,  79000],
       [    27,  54000],
       [    30, 135000],
       [    31,  89000],
       [    24,  32000],
       [    18,  44000],
       [    29,  83000],
       [    35,  23000],
       [    27,  58000],
       [    24,  55000],
       [    23,  48000],
       [    28,  79000],
       [    22,  18000],
       [    32, 117000],
       [    27,  20000],
       [    25,  87000],
       [    23,  66000],
       [    32, 120000],
```

```
       [    59,  83000],
       [    24,  58000],
       [    24,  19000],
       [    23,  82000],
       [    22,  63000],
       [    31,  68000],
       [    25,  80000],
       [    24,  27000],
       [    20,  23000],
       [    33, 113000],
       [    32,  18000],
       [    34, 112000],
       [    18,  52000],
       [    22,  27000],
       [    28,  87000],
       [    26,  17000],
       [    30,  80000],
       [    39,  42000],
       [    20,  49000],
       [    35,  88000],
       [    30,  62000],
       [    31, 118000],
       [    24,  55000],
       [    28,  85000],
       [    26,  81000],
       [    35,  50000],
       [    22,  81000],
       [    30, 116000],
       [    26,  15000],
       [    29,  28000],
       [    29,  83000],
       [    35,  44000],
       [    35,  25000],
       [    28, 123000],
       [    35,  73000],
       [    28,  37000],
       [    27,  88000],
       [    28,  59000],
       [    32,  86000],
       [    33, 149000],
       [    19,  21000],
       [    21,  72000],
       [    26,  35000],
       [    27,  89000],
       [    26,  86000],
       [    38,  80000],
       [    39,  71000],
       [    37,  71000],
       [    38,  61000],
```

```
       [    37,  55000],
       [    42,  80000],
       [    40,  57000],
       [    35,  75000],
       [    36,  52000],
       [    40,  59000],
       [    41,  59000],
       [    36,  75000],
       [    37,  72000],
       [    40,  75000],
       [    35,  53000],
       [    41,  51000],
       [    39,  61000],
       [    42,  65000],
       [    26,  32000],
       [    30,  17000],
       [    26,  84000],
       [    31,  58000],
       [    33,  31000],
       [    30,  87000],
       [    21,  68000],
       [    28,  55000],
       [    23,  63000],
       [    20,  82000],
       [    30, 107000],
       [    28,  59000],
       [    19,  25000],
       [    19,  85000],
       [    18,  68000],
       [    35,  59000],
       [    30,  89000],
       [    34,  25000],
       [    24,  89000],
       [    27,  96000],
       [    41,  30000],
       [    29,  61000],
       [    20,  74000],
       [    26,  15000],
       [    41,  45000],
       [    31,  76000],
       [    36,  50000],
       [    40,  47000],
       [    31,  15000],
       [    46,  59000],
       [    29,  75000],
       [    26,  30000],
       [    32, 135000],
       [    32, 100000],
       [    25,  90000],
```

```
       [    37,  33000],
       [    35,  38000],
       [    33,  69000],
       [    18,  86000],
       [    22,  55000],
       [    35,  71000],
       [    29, 148000],
       [    29,  47000],
       [    21,  88000],
       [    34, 115000],
       [    26, 118000],
       [    34,  43000],
       [    34,  72000],
       [    23,  28000],
       [    35,  47000],
       [    25,  22000],
       [    24,  23000],
       [    31,  34000],
       [    26,  16000],
       [    31,  71000],
       [    32, 117000],
       [    33,  43000],
       [    33,  60000],
       [    31,  66000],
       [    20,  82000],
       [    33,  41000],
       [    35,  72000],
       [    28,  32000],
       [    24,  84000],
       [    19,  26000],
       [    29,  43000],
       [    19,  70000],
       [    28,  89000],
       [    34,  43000],
       [    30,  79000],
       [    20,  36000],
       [    26,  80000],
       [    35,  22000],
       [    35,  39000],
       [    49,  74000],
       [    39, 134000],
       [    41,  71000],
       [    58, 101000],
       [    47,  47000],
       [    55, 130000],
       [    52, 114000],
       [    40, 142000],
       [    46,  22000],
       [    48,  96000],
```

```
       [    52, 150000],
       [    59,  42000],
       [    35,  58000],
       [    47,  43000],
       [    60, 108000],
       [    49,  65000],
       [    40,  78000],
       [    46,  96000],
       [    59, 143000],
       [    41,  80000],
       [    35,  91000],
       [    37, 144000],
       [    60, 102000],
       [    35,  60000],
       [    37,  53000],
       [    36, 126000],
       [    56, 133000],
       [    40,  72000],
       [    42,  80000],
       [    35, 147000],
       [    39,  42000],
       [    40, 107000],
       [    49,  86000],
       [    38, 112000],
       [    46,  79000],
       [    40,  57000],
       [    37,  80000],
       [    46,  82000],
       [    53, 143000],
       [    42, 149000],
       [    38,  59000],
       [    50,  88000],
       [    56, 104000],
       [    41,  72000],
       [    51, 146000],
       [    35,  50000],
       [    57, 122000],
       [    41,  52000],
       [    35,  97000],
       [    44,  39000],
       [    37,  52000],
       [    48, 134000],
       [    37, 146000],
       [    50,  44000],
       [    52,  90000],
       [    41,  72000],
       [    40,  57000],
       [    58,  95000],
       [    45, 131000],
```

```
       [    35,  77000],
       [    36, 144000],
       [    55, 125000],
       [    35,  72000],
       [    48,  90000],
       [    42, 108000],
       [    40,  75000],
       [    37,  74000],
       [    47, 144000],
       [    40,  61000],
       [    43, 133000],
       [    59,  76000],
       [    60,  42000],
       [    39, 106000],
       [    57,  26000],
       [    57,  74000],
       [    38,  71000],
       [    49,  88000],
       [    52,  38000],
       [    50,  36000],
       [    59,  88000],
       [    35,  61000],
       [    37,  70000],
       [    52,  21000],
       [    48, 141000],
       [    37,  93000],
       [    37,  62000],
       [    48, 138000],
       [    41,  79000],
       [    37,  78000],
       [    39, 134000],
       [    49,  89000],
       [    55,  39000],
       [    37,  77000],
       [    35,  57000],
       [    36,  63000],
       [    42,  73000],
       [    43, 112000],
       [    45,  79000],
       [    46, 117000],
       [    58,  38000],
       [    48,  74000],
       [    37, 137000],
       [    37,  79000],
       [    40,  60000],
       [    42,  54000],
       [    51, 134000],
       [    47, 113000],
       [    36, 125000],
```

```
       [    38,  50000],
       [    42,  70000],
       [    39,  96000],
       [    38,  50000],
       [    49, 141000],
       [    39,  79000],
       [    39,  75000],
       [    54, 104000],
       [    35,  55000],
       [    45,  32000],
       [    36,  60000],
       [    52, 138000],
       [    53,  82000],
       [    41,  52000],
       [    48,  30000],
       [    48, 131000],
       [    41,  60000],
       [    41,  72000],
       [    42,  75000],
       [    36, 118000],
       [    47, 107000],
       [    38,  51000],
       [    48, 119000],
       [    42,  65000],
       [    40,  65000],
       [    57,  60000],
       [    36,  54000],
       [    58, 144000],
       [    35,  79000],
       [    38,  55000],
       [    39, 122000],
       [    53, 104000],
       [    35,  75000],
       [    38,  65000],
       [    47,  51000],
       [    47, 105000],
       [    41,  63000],
       [    53,  72000],
       [    54, 108000],
       [    39,  77000],
       [    38,  61000],
       [    38, 113000],
       [    37,  75000],
       [    42,  90000],
       [    37,  57000],
       [    36,  99000],
       [    60,  34000],
       [    54,  70000],
       [    41,  72000],
```

```
       [    40,  71000],
       [    42,  54000],
       [    43, 129000],
       [    53,  34000],
       [    47,  50000],
       [    42,  79000],
       [    42, 104000],
       [    59,  29000],
       [    58,  47000],
       [    46,  88000],
       [    38,  71000],
       [    54,  26000],
       [    60,  46000],
       [    60,  83000],
       [    39,  73000],
       [    59, 130000],
       [    37,  80000],
       [    46,  32000],
       [    46,  74000],
       [    42,  53000],
       [    41,  87000],
       [    58,  23000],
       [    42,  64000],
       [    48,  33000],
       [    44, 139000],
       [    49,  28000],
       [    57,  33000],
       [    56,  60000],
       [    49,  39000],
       [    39,  71000],
       [    47,  34000],
       [    48,  35000],
       [    48,  33000],
       [    47,  23000],
       [    45,  45000],
       [    60,  42000],
       [    39,  59000],
       [    46,  41000],
       [    51,  23000],
       [    50,  20000],
       [    36,  33000],
       [    49,  36000]])

label

array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
1,
       1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
```

```
0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0,
1,
       0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1,
0,
       1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1,
0,
       1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0,
1,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0,
1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1,
1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1,
0,
       1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0,
1,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0,
1,
       1, 1, 0, 1])

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

for i in range(1,401):

x_train,x_test,y_train,y_test=train_test_split(features,label,test_siz
e=0.3,random_state=i)
    model=LogisticRegression()
    model.fit(x_train,y_train)
    train_score=model.score(x_train,y_train)
    test_score=model.score(x_test,y_test)
    if test_score>train_score:
        print("Test {} Train{} Random State
{}".format(test_score,train_score,i))

Test 0.8833333333333333 Train0.8285714285714286 Random State 4
Test 0.8666666666666667 Train0.8464285714285714 Random State 5
```

```
Test 0.875 Train0.8464285714285714 Random State 6
Test 0.875 Train0.8392857142857143 Random State 7
Test 0.875 Train0.8464285714285714 Random State 10
Test 0.8583333333333333 Train0.8285714285714286 Random State 13
Test 0.8666666666666667 Train0.8535714285714285 Random State 14
Test 0.8833333333333333 Train0.8464285714285714 Random State 15
Test 0.8833333333333333 Train0.8464285714285714 Random State 16
Test 0.8583333333333333 Train0.8535714285714285 Random State 17
Test 0.8833333333333333 Train0.8285714285714286 Random State 18
Test 0.85 Train0.8464285714285714 Random State 19
Test 0.8416666666666667 Train0.825 Random State 20
Test 0.8583333333333333 Train0.8357142857142857 Random State 21
Test 0.875 Train0.8464285714285714 Random State 22
Test 0.8666666666666667 Train0.8464285714285714 Random State 24
Test 0.8583333333333333 Train0.8285714285714286 Random State 26
Test 0.875 Train0.8214285714285714 Random State 27
Test 0.8833333333333333 Train0.8464285714285714 Random State 31
Test 0.8583333333333333 Train0.8464285714285714 Random State 33
Test 0.8416666666666667 Train0.8392857142857143 Random State 35
Test 0.8833333333333333 Train0.8357142857142857 Random State 38
Test 0.85 Train0.8321428571428572 Random State 42
Test 0.85 Train0.8357142857142857 Random State 47
Test 0.8666666666666667 Train0.8321428571428572 Random State 48
Test 0.8583333333333333 Train0.85 Random State 50
Test 0.8666666666666667 Train0.8321428571428572 Random State 51
Test 0.8416666666666667 Train0.8357142857142857 Random State 52
Test 0.8916666666666667 Train0.8285714285714286 Random State 54
Test 0.8583333333333333 Train0.8464285714285714 Random State 57
Test 0.85 Train0.8428571428571429 Random State 59
Test 0.8916666666666667 Train0.85 Random State 61
Test 0.8666666666666667 Train0.8285714285714286 Random State 65
Test 0.8833333333333333 Train0.8392857142857143 Random State 68
Test 0.8666666666666667 Train0.8321428571428572 Random State 72
Test 0.8583333333333333 Train0.8321428571428572 Random State 73
Test 0.875 Train0.8464285714285714 Random State 74
Test 0.8666666666666667 Train0.85 Random State 75
Test 0.9083333333333333 Train0.8107142857142857 Random State 76
Test 0.8833333333333333 Train0.8285714285714286 Random State 77
Test 0.8666666666666667 Train0.8357142857142857 Random State 79
Test 0.8666666666666667 Train0.8642857142857143 Random State 81
Test 0.85 Train0.8285714285714286 Random State 82
Test 0.8666666666666667 Train0.8464285714285714 Random State 83
Test 0.8833333333333333 Train0.8428571428571429 Random State 85
Test 0.8583333333333333 Train0.8535714285714285 Random State 86
Test 0.8416666666666667 Train0.8392857142857143 Random State 88
Test 0.9 Train0.8464285714285714 Random State 90
Test 0.8666666666666667 Train0.85 Random State 99
Test 0.8583333333333333 Train0.85 Random State 102
Test 0.8666666666666667 Train0.8428571428571429 Random State 104
```

```
Test 0.8583333333333333 Train0.85 Random State 105
Test 0.875 Train0.8357142857142857 Random State 106
Test 0.8583333333333333 Train0.8392857142857143 Random State 107
Test 0.8583333333333333 Train0.85 Random State 108
Test 0.9 Train0.8321428571428572 Random State 112
Test 0.875 Train0.8392857142857143 Random State 115
Test 0.875 Train0.8428571428571429 Random State 116
Test 0.8583333333333333 Train0.8321428571428572 Random State 119
Test 0.9166666666666666 Train0.8285714285714286 Random State 120
Test 0.85 Train0.8464285714285714 Random State 123
Test 0.8666666666666667 Train0.8607142857142858 Random State 125
Test 0.875 Train0.8428571428571429 Random State 128
Test 0.8583333333333333 Train0.8535714285714285 Random State 130
Test 0.8916666666666667 Train0.8285714285714286 Random State 133
Test 0.8583333333333333 Train0.8428571428571429 Random State 135
Test 0.85 Train0.8321428571428572 Random State 137
Test 0.8833333333333333 Train0.8214285714285714 Random State 141
Test 0.8666666666666667 Train0.85 Random State 143
Test 0.8666666666666667 Train0.8607142857142858 Random State 145
Test 0.875 Train0.8428571428571429 Random State 148
Test 0.9 Train0.825 Random State 150
Test 0.85 Train0.8392857142857143 Random State 151
Test 0.9166666666666666 Train0.8321428571428572 Random State 152
Test 0.8666666666666667 Train0.8357142857142857 Random State 153
Test 0.8833333333333333 Train0.8392857142857143 Random State 155
Test 0.8833333333333333 Train0.8428571428571429 Random State 156
Test 0.8666666666666667 Train0.8285714285714286 Random State 159
Test 0.8583333333333333 Train0.8428571428571429 Random State 161
Test 0.8666666666666667 Train0.8464285714285714 Random State 162
Test 0.8833333333333333 Train0.825 Random State 163
Test 0.8416666666666667 Train0.8392857142857143 Random State 164
Test 0.85 Train0.8428571428571429 Random State 166
Test 0.8583333333333333 Train0.8321428571428572 Random State 167
Test 0.8583333333333333 Train0.85 Random State 169
Test 0.85 Train0.8357142857142857 Random State 173
Test 0.8333333333333334 Train0.8285714285714286 Random State 174
Test 0.8666666666666667 Train0.8642857142857143 Random State 178
Test 0.875 Train0.825 Random State 180
Test 0.8666666666666667 Train0.8464285714285714 Random State 184
Test 0.8583333333333333 Train0.8321428571428572 Random State 185
Test 0.9 Train0.8214285714285714 Random State 186
Test 0.8666666666666667 Train0.8428571428571429 Random State 187
Test 0.8833333333333333 Train0.8464285714285714 Random State 192
Test 0.8833333333333333 Train0.825 Random State 193
Test 0.8583333333333333 Train0.8428571428571429 Random State 195
Test 0.8916666666666667 Train0.825 Random State 196
Test 0.8916666666666667 Train0.825 Random State 197
Test 0.85 Train0.8357142857142857 Random State 198
Test 0.8583333333333333 Train0.8428571428571429 Random State 200
```

Test 0.875 Train0.8321428571428572 Random State 202
Test 0.875 Train0.8285714285714286 Random State 203
Test 0.85 Train0.8357142857142857 Random State 206
Test 0.8666666666666667 Train0.85 Random State 213
Test 0.85 Train0.8392857142857143 Random State 214
Test 0.85 Train0.8428571428571429 Random State 217
Test 0.8416666666666667 Train0.8392857142857143 Random State 218
Test 0.925 Train0.8178571428571428 Random State 220
Test 0.9083333333333333 Train0.8214285714285714 Random State 223
Test 0.875 Train0.8285714285714286 Random State 227
Test 0.8916666666666667 Train0.825 Random State 228
Test 0.8916666666666667 Train0.8428571428571429 Random State 229
Test 0.8666666666666667 Train0.8464285714285714 Random State 232
Test 0.8666666666666667 Train0.85 Random State 233
Test 0.8583333333333333 Train0.8357142857142857 Random State 234
Test 0.8583333333333333 Train0.8321428571428572 Random State 235
Test 0.875 Train0.8285714285714286 Random State 237
Test 0.8416666666666667 Train0.8357142857142857 Random State 240
Test 0.8666666666666667 Train0.8464285714285714 Random State 242
Test 0.8583333333333333 Train0.8321428571428572 Random State 243
Test 0.8833333333333333 Train0.85 Random State 244
Test 0.8833333333333333 Train0.8464285714285714 Random State 245
Test 0.8666666666666667 Train0.85 Random State 246
Test 0.9083333333333333 Train0.8357142857142857 Random State 247
Test 0.8583333333333333 Train0.8535714285714285 Random State 248
Test 0.85 Train0.8428571428571429 Random State 251
Test 0.875 Train0.8464285714285714 Random State 252
Test 0.8666666666666667 Train0.8464285714285714 Random State 255
Test 0.875 Train0.8392857142857143 Random State 256
Test 0.8583333333333333 Train0.8285714285714286 Random State 257
Test 0.8833333333333333 Train0.8428571428571429 Random State 260
Test 0.8583333333333333 Train0.85 Random State 269
Test 0.8583333333333333 Train0.8357142857142857 Random State 270
Test 0.8583333333333333 Train0.8428571428571429 Random State 276
Test 0.8916666666666667 Train0.8285714285714286 Random State 277
Test 0.8416666666666667 Train0.8392857142857143 Random State 282
Test 0.8583333333333333 Train0.8428571428571429 Random State 283
Test 0.875 Train0.8214285714285714 Random State 285
Test 0.85 Train0.8428571428571429 Random State 286
Test 0.8583333333333333 Train0.85 Random State 287
Test 0.8583333333333333 Train0.8321428571428572 Random State 288
Test 0.875 Train0.8285714285714286 Random State 290
Test 0.8833333333333333 Train0.8392857142857143 Random State 292
Test 0.8583333333333333 Train0.8321428571428572 Random State 294
Test 0.8583333333333333 Train0.8392857142857143 Random State 297
Test 0.9083333333333333 Train0.8428571428571429 Random State 302
Test 0.875 Train0.85 Random State 303
Test 0.875 Train0.8428571428571429 Random State 305
Test 0.8916666666666667 Train0.8428571428571429 Random State 306

```
Test 0.875 Train0.8428571428571429 Random State 308
Test 0.85 Train0.8464285714285714 Random State 311
Test 0.875 Train0.8321428571428572 Random State 314
Test 0.8916666666666667 Train0.8321428571428572 Random State 315
Test 0.8833333333333333 Train0.85 Random State 317
Test 0.85 Train0.8428571428571429 Random State 318
Test 0.8583333333333333 Train0.8321428571428572 Random State 319
Test 0.8666666666666667 Train0.8428571428571429 Random State 321
Test 0.9083333333333333 Train0.8321428571428572 Random State 322
Test 0.8583333333333333 Train0.8392857142857143 Random State 331
Test 0.85 Train0.8392857142857143 Random State 333
Test 0.9 Train0.8392857142857143 Random State 336
Test 0.8666666666666667 Train0.8357142857142857 Random State 337
Test 0.875 Train0.8285714285714286 Random State 343
Test 0.9 Train0.8357142857142857 Random State 351
Test 0.8666666666666667 Train0.8321428571428572 Random State 352
Test 0.9 Train0.8178571428571428 Random State 354
Test 0.8666666666666667 Train0.8428571428571429 Random State 357
Test 0.8666666666666667 Train0.8357142857142857 Random State 358
Test 0.85 Train0.8464285714285714 Random State 361
Test 0.875 Train0.8285714285714286 Random State 362
Test 0.925 Train0.8214285714285714 Random State 363
Test 0.8833333333333333 Train0.8285714285714286 Random State 366
Test 0.8916666666666667 Train0.85 Random State 369
Test 0.8916666666666667 Train0.8392857142857143 Random State 371
Test 0.9 Train0.8321428571428572 Random State 376
Test 0.9083333333333333 Train0.8142857142857143 Random State 377
Test 0.8833333333333333 Train0.8428571428571429 Random State 378
Test 0.8833333333333333 Train0.8428571428571429 Random State 379
Test 0.8583333333333333 Train0.8357142857142857 Random State 381
Test 0.8583333333333333 Train0.8321428571428572 Random State 382
Test 0.8833333333333333 Train0.8535714285714285 Random State 386
Test 0.8416666666666667 Train0.8357142857142857 Random State 388
Test 0.8583333333333333 Train0.8535714285714285 Random State 390
Test 0.875 Train0.825 Random State 394
Test 0.875 Train0.8535714285714285 Random State 399
Test 0.875 Train0.8428571428571429 Random State 400
```

```
x_train,x_test,y_train,y_test=train_test_split(features,label,test_siz
e=0.2,random_state=42)
finalModel=LogisticRegression()
finalModel.fit(x_train,y_train)
```

```
LogisticRegression()
```

```
print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))
```

```
0.8375
0.8875
```

```
from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))
```

```
              precision    recall  f1-score   support

           0       0.85      0.93      0.89       257
           1       0.85      0.70      0.77       143

    accuracy                           0.85       400
   macro avg       0.85      0.81      0.83       400
weighted avg       0.85      0.85      0.84       400
```

Name: Thamizhselvan.V Register Number: 250701780

Experiment Name: K-Means Clustering

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

df=pd.read_csv('Mall_Customers.csv')

df.info()
```

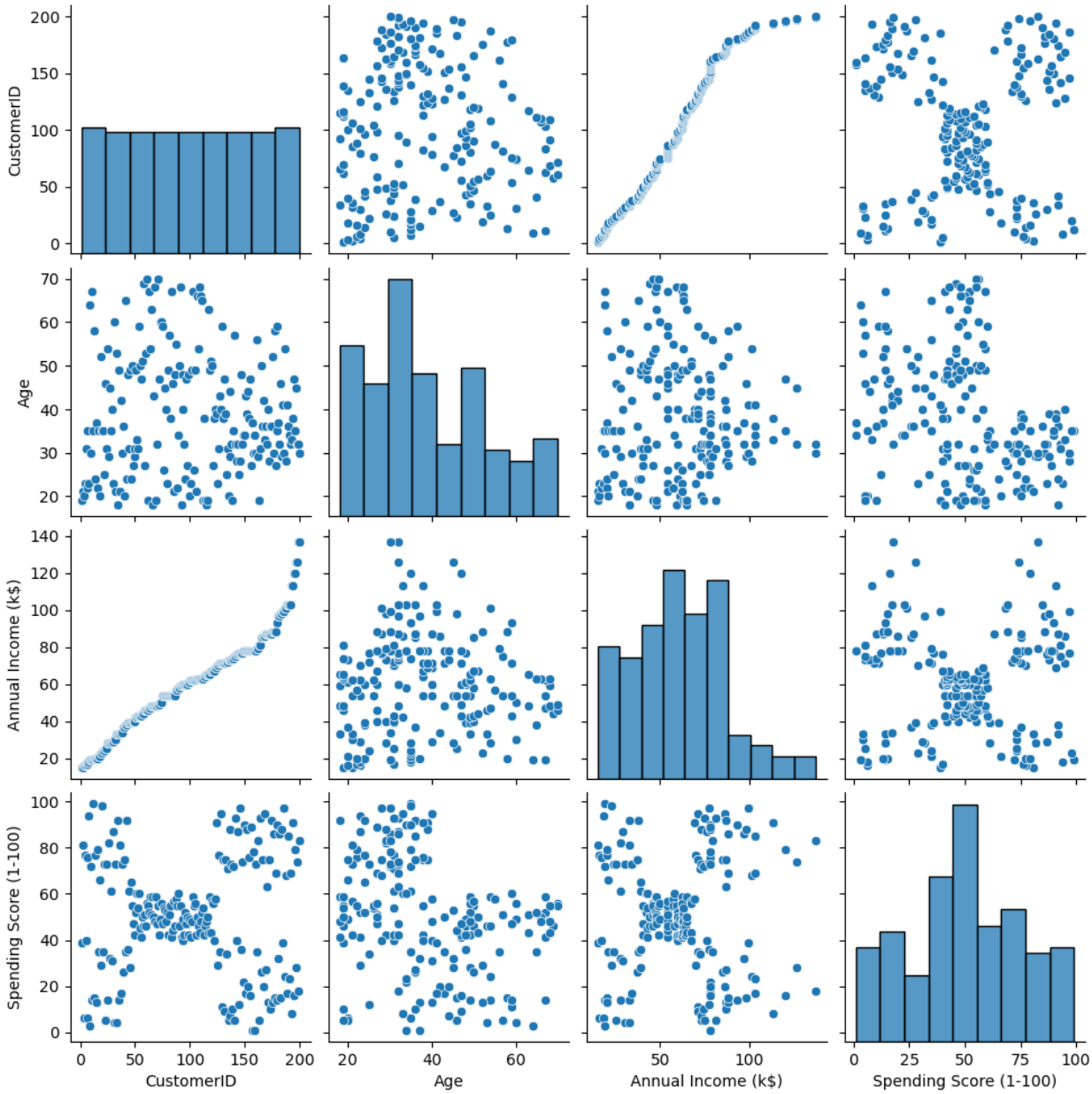
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   CustomerID              200 non-null    int64  
 1   Gender                  200 non-null    object 
 2   Age                     200 non-null    int64  
 3   Annual Income (k$)      200 non-null    int64  
 4   Spending Score (1-100)  200 non-null    int64  
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

```
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
sns.pairplot(df)
```

```
<seaborn.axisgrid.PairGrid at 0x1f67ce53230>
```




```
features=df.iloc[:,[3,4]].values

from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)

C:\Users\swath\anaconda3\Lib\site-packages\sklearn\cluster\
_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on
Windows with MKL, when there are less chunks than available threads.
You can avoid it by setting the environment variable
```

```
OMP_NUM_THREADS=1.
  warnings.warn(

KMeans(n_clusters=5)

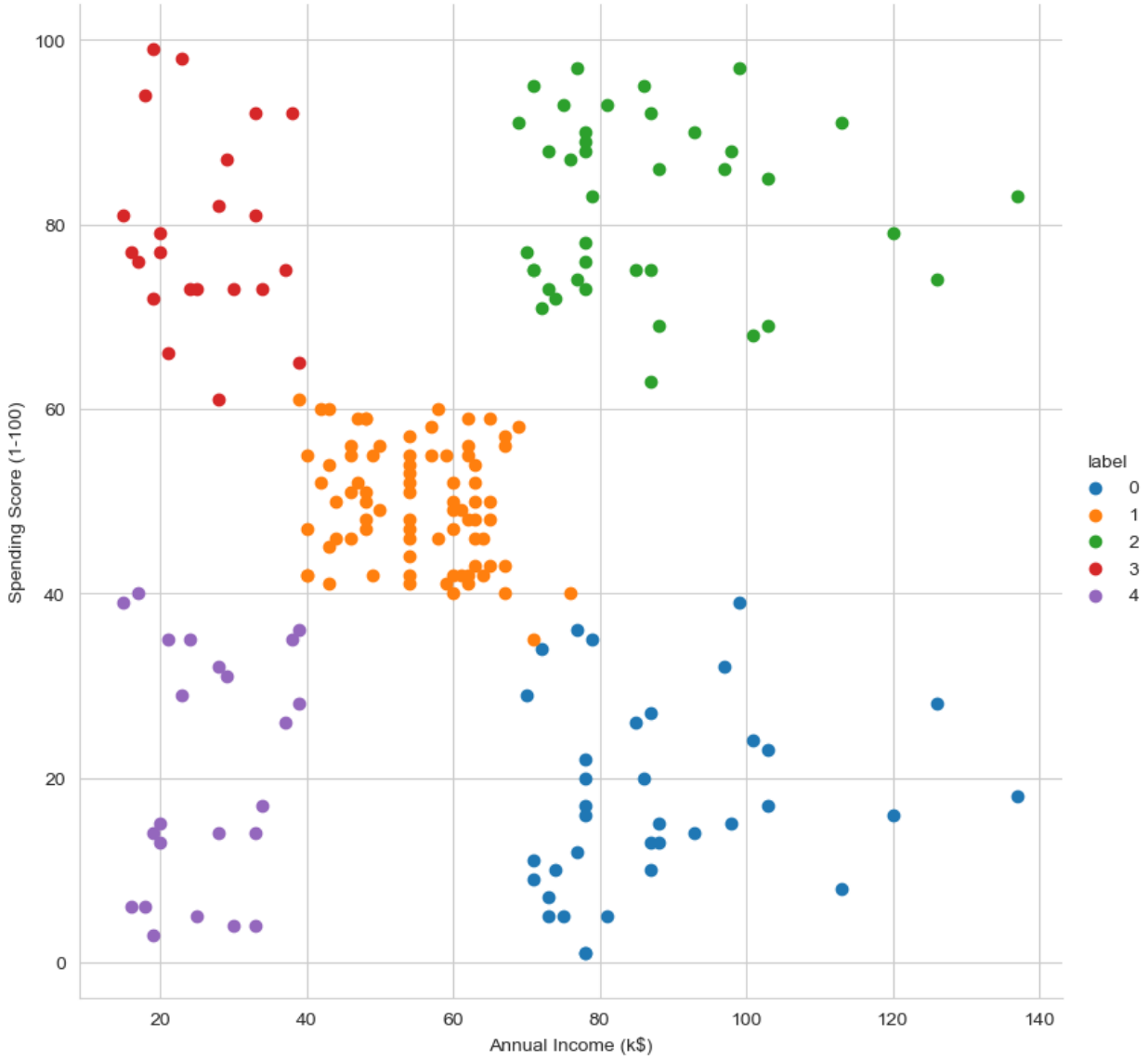
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()

C:\Users\swath\AppData\Local\Temp\ipykernel_5828\470183701.py:2:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  Final['label']=model.predict(features)

   Annual Income (k$)  Spending Score (1-100)  label
0                  15                      39      4
1                  15                      81      3
2                  16                       6      4
3                  16                      77      3
4                  17                      40      4

sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend();
plt.show()
```




```
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
    model=KMeans(n_clusters=i)
model.fit(features_el)
wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```

```
  Cell In[11], line 5
    model=KMeans(n_clusters=i)
    ^
IndentationError: expected an indented block after 'for' statement on
line 4
```

Name: Thamizhselvan.VRegister Number: 250701780

```
import numpy as np
import pandas as pd

df=pd.read_csv('Iris.csv')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object 
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
df.variety.value_counts()
```

```
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

```
df.head()
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

```
features=df.iloc[:,:-1].values
label=df.iloc[:,4].values

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2
,random_state=42)
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)
```

```
KNeighborsClassifier()
```

```
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
```

```
0.9666666666666667
1.0
```

```
from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))
```

```
array([[50,  0,  0],
       [ 0, 47,  3],
       [ 0,  1, 49]])
```

Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Z Test, T Test, F Test, ANOVA Test

T - Test

A sample of 10 students scored the following marks in an exam: [72, 68, 75, 70, 74, 69, 71, 73, 70, 72] We want to test whether the average mark = 70 (μ0 = 70) at 5% significance level using python

```
import numpy as np
from scipy import stats
marks = np.array([72, 68, 75, 70, 74, 69, 71, 73, 70, 72])
mu_0 = 70
t_stat, p_value = stats.ttest_1samp(marks, mu_0)
print(f"T-statistic: {t_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different
from 70.")
else:
    print("Fail to Reject Null Hypothesis → No significant
difference.")
```

```
T-statistic: 1.993
P-value: 0.0774
Fail to Reject Null Hypothesis → No significant difference.
```

Z - Test

```
import numpy as np
from math import sqrt
from scipy.stats import norm
x_bar = 51.2
mu_0 = 50
sigma = 3
n = 36
z_stat = (x_bar - mu_0) / (sigma / sqrt(n))
p_value = 2 * (1 - norm.cdf(abs(z_stat)))
print(f"Z-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different
from 50 g.")
else:
    print("Fail to Reject Null Hypothesis → No significant
difference.")
```

```
Z-statistic: 2.400
P-value: 0.0164
Reject Null Hypothesis → Mean is significantly different from 50 g.
```

Anova Test

Three fertilizers (A, B, C) were tested on crop yield (in kg). Is there a significant difference among fertilizers? (Use α = 0.05)

Fertilizer Yields

A 20, 22, 23

B 19, 20, 18

C 25, 27, 26

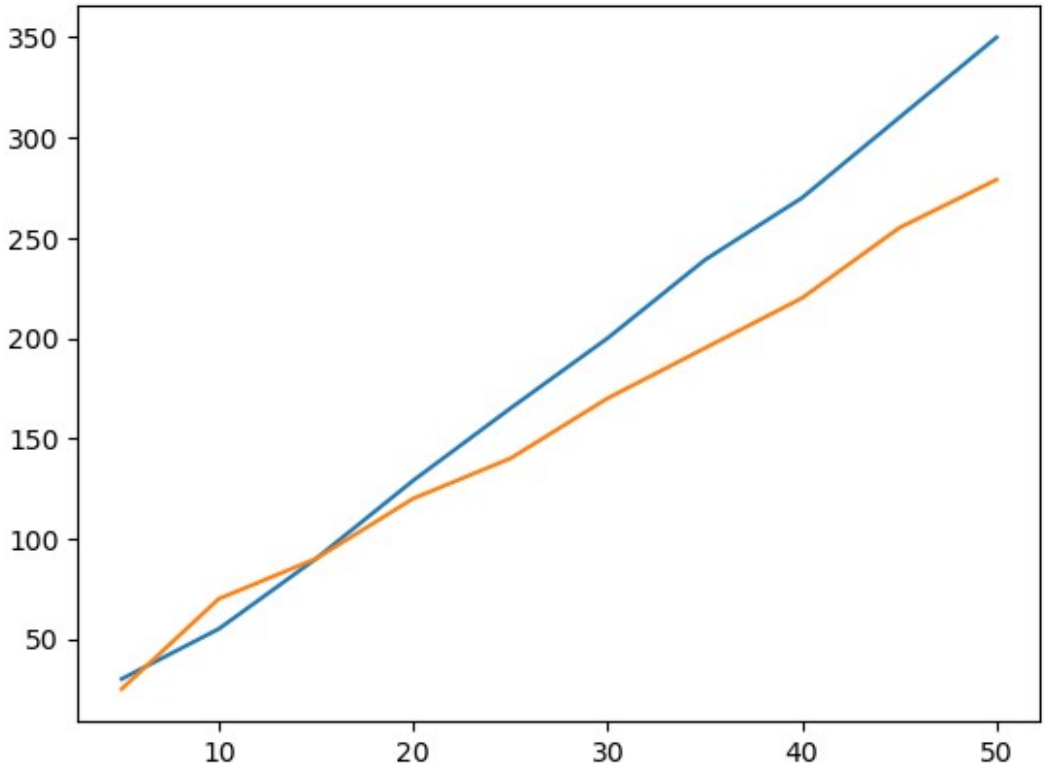
```
import numpy as np
from scipy import stats
A=[20,22,23]
B=[19,20,18]
C=[25,27,26]
f_stat,p_value=stats.f_oneway(A,B,C)
print(f"F-statistic: {f_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha=0.05
if p_value<alpha:
    print("Reject Null Hypothesis-Means are significantly different.")
else:
    print("Fail to Reject Null Hypo
```

Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Data Visualization Using Line Plot, Bar Chart, Pie Chart

a.Conduct an experiment to show data visualization using line plot

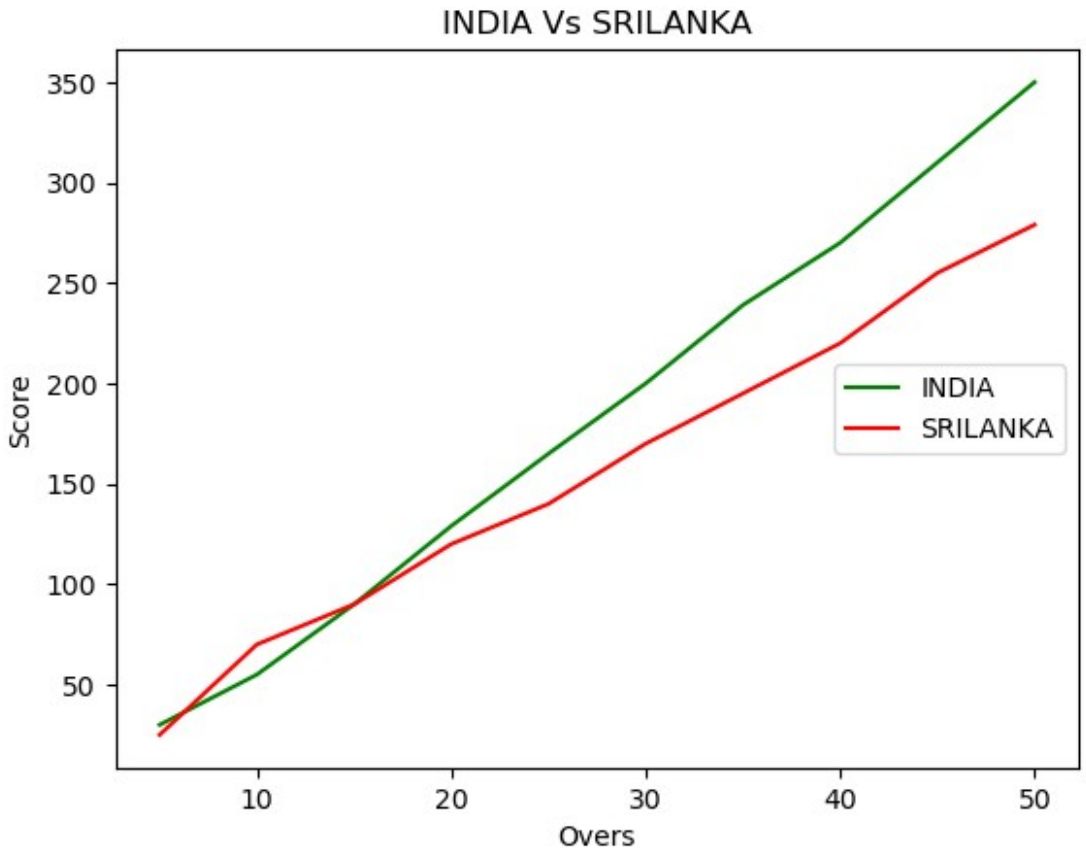
```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```




```
C:\Users\REC\AppData\Local\Temp\ipykernel_12032\506286342.py:11:
UserWarning: No artists with labels found to put in legend.  Note that
artists whose label start with an underscore are ignored when legend()
```

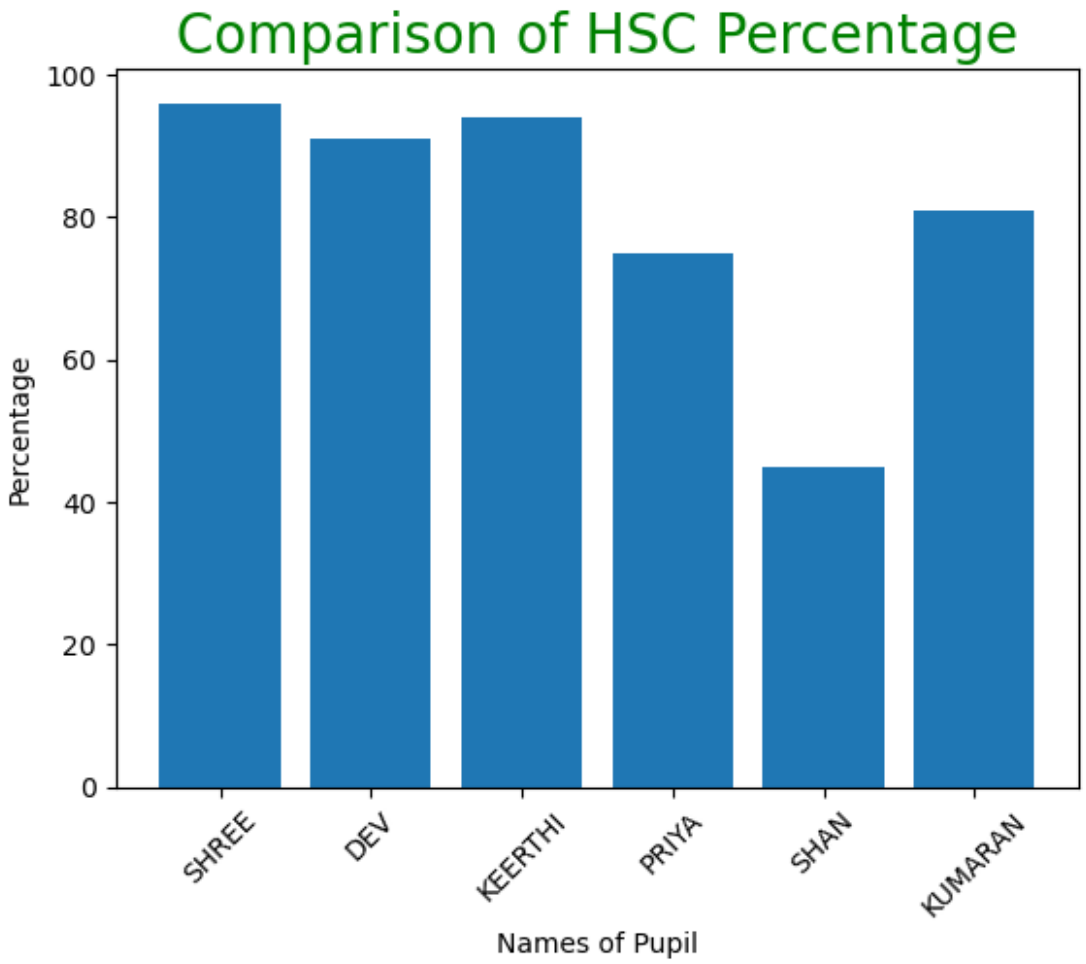
```
is called with no argument.
  cricket.legend()

<matplotlib.legend.Legend at 0x1659703fc50>
```



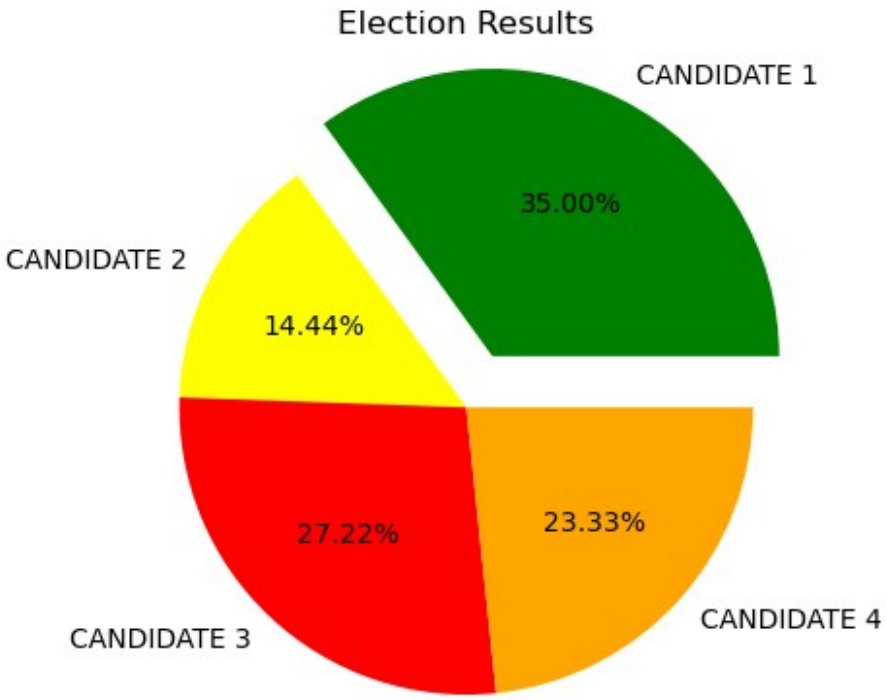
b.Conduct an experiment to show data visualization using bar chart

```
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20,
color="green")
hscmark.show()
```



c.Conduct an experiment to show data visualization using pie chart

```
import matplotlib.pyplot as election
# Election data
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)
# Plotting the pie chart
election.pie(Votes, labels=labels, colors=colors, explode=explode,
autopct='%0.2f%%')
election.title('Election Results')
election.show()
```


Election Results
CANDIDATE 1
35.00%
CANDIDATE 2
14.44%
27.22%
23.33%
CANDIDATE 3
CANDIDATE 4

Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Word Frequency in Text Processing

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
# Download the gutenberg corpus if not already installed
nltk.download('gutenberg')
nltk.download('punkt') # Ensure the punkt tokenizer models are also
downloaded
# Load the text from the gutenberg corpus
sample = gutenberg.raw("austen-emma.txt") # Change to an existing text
from the corpus
# Tokenize the sample text
token = word_tokenize(sample)
# Create a list of the first 50 tokens
wlist = []
for i in range(50):
    wlist.append(token[i])
# Calculate the frequency of each word in the list
wordfreq = [wlist.count(w) for w in wlist]
# Print the word-frequency pairs
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1),
('1816',
1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2),
('Emma
', 2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5),
('clever'
, 1), (',', 5), ('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a',
1
), ('comfortable', 1), ('home', 1), ('and', 3), ('happy', 1),
('disposi
tion', 1), (',', 5), ('seemed', 1), ('to', 1), ('unite', 1), ('some',
1
), ('of', 2), ('the', 2), ('best', 1), ('blessings', 1), ('of', 2),
('e
xistence', 1), (';', 1), ('and', 3), ('had', 1), ('lived', 1),
('nearly
', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2),
('world'
, 1), ('with', 2)]
```

Name:ThamizhselvanR Register Number:250701780

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create a sample diabetes dataset
data = {
    'Pregnancies': [6, 1, 8, 1, 0, 5, 3, 10, 2, 4],
    'Glucose': [148, 85, 183, 89, 137, 116, 78, 115, 197, 110],
    'BloodPressure': [72, 66, 64, 66, 40, 74, 50, 0, 70, 92],
    'SkinThickness': [35, 29, 0, 23, 35, 0, 32, 0, 45, 0],
    'Insulin': [0, 0, 0, 94, 168, 0, 88, 0, 543, 0],
    'BMI': [33.6, 26.6, 23.3, 28.1, 43.1, 25.6, 31.0, 35.3, 30.5, 37.6],
    'DiabetesPedigreeFunction': [0.627, 0.351, 0.672, 0.167, 2.288, 0.201, 0.248, 0
    'Age': [50, 31, 32, 21, 33, 30, 26, 29, 53, 30],
    'Outcome': [1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
}

diabetes_df = pd.DataFrame(data)

# 1. Display the first five rows
print("First Five Rows:")
print(diabetes_df.head())

# 2. Display the last five rows
print("\nLast Five Rows:")
print(diabetes_df.tail())

# 3. Display dataset information
print("\nDataset Information:")
diabetes_df.info()

# 4. Display the number of rows and columns
print("\nDataset Shape:")
print(diabetes_df.shape)

# 5. Display column names
print("\nColumn Names:")
print(diabetes_df.columns)

# 6. Check for missing values
print("\nMissing Values:")
print(diabetes_df.isnull().sum())

# 7. Display statistical summary
print("\nStatistical Summary:")
print(diabetes_df.describe())

# 8. Check duplicate rows
print("\nNumber of Duplicate Rows:")
print(diabetes_df.duplicated().sum())

# 9. Visualize the correlation between columns
plt.figure(figsize=(10, 6))
```

```
sns.heatmap(diabetes_df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Heatmap of Diabetes Dataset")
plt.show()

# 10. Visualize the outcome distribution
sns.countplot(data=diabetes_df, x='Outcome')
plt.title("Diabetes Outcome Distribution")
plt.xlabel("Outcome (0 = No Diabetes, 1 = Diabetes)")
plt.ylabel("Count")
plt.show()
```

```
First Five Rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

Last Five Rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
5            5      116             74              0        0  25.6
6            3       78             50             32       88  31.0
7           10      115              0              0        0  35.3
8            2      197             70             45      543  30.5
9            4      110             92              0        0  37.6

   DiabetesPedigreeFunction  Age  Outcome
5                     0.201   30        0
6                     0.248   26        1
7                     0.134   29        0
8                     0.158   53        1
9                     0.191   30        0

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               10 non-null     int64
 1   Glucose                   10 non-null     int64
 2   BloodPressure             10 non-null     int64
 3   SkinThickness             10 non-null     int64
 4   Insulin                   10 non-null     int64
 5   BMI                       10 non-null     float64
 6   DiabetesPedigreeFunction  10 non-null     float64
 7   Age                       10 non-null     int64
 8   Outcome                   10 non-null     int64
dtypes: float64(2), int64(7)
memory usage: 852.0 bytes

Dataset Shape:
(10, 9)

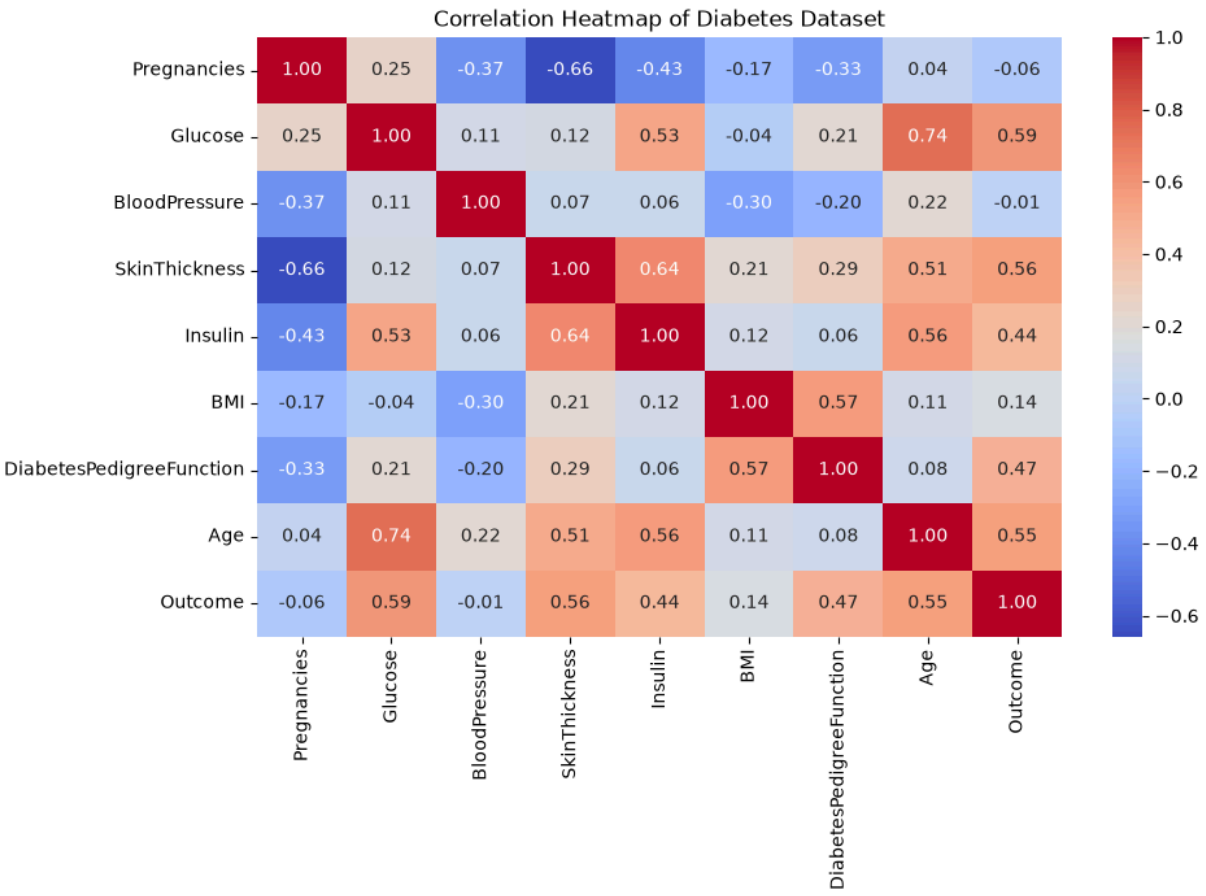
Column Names:
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='str')
```

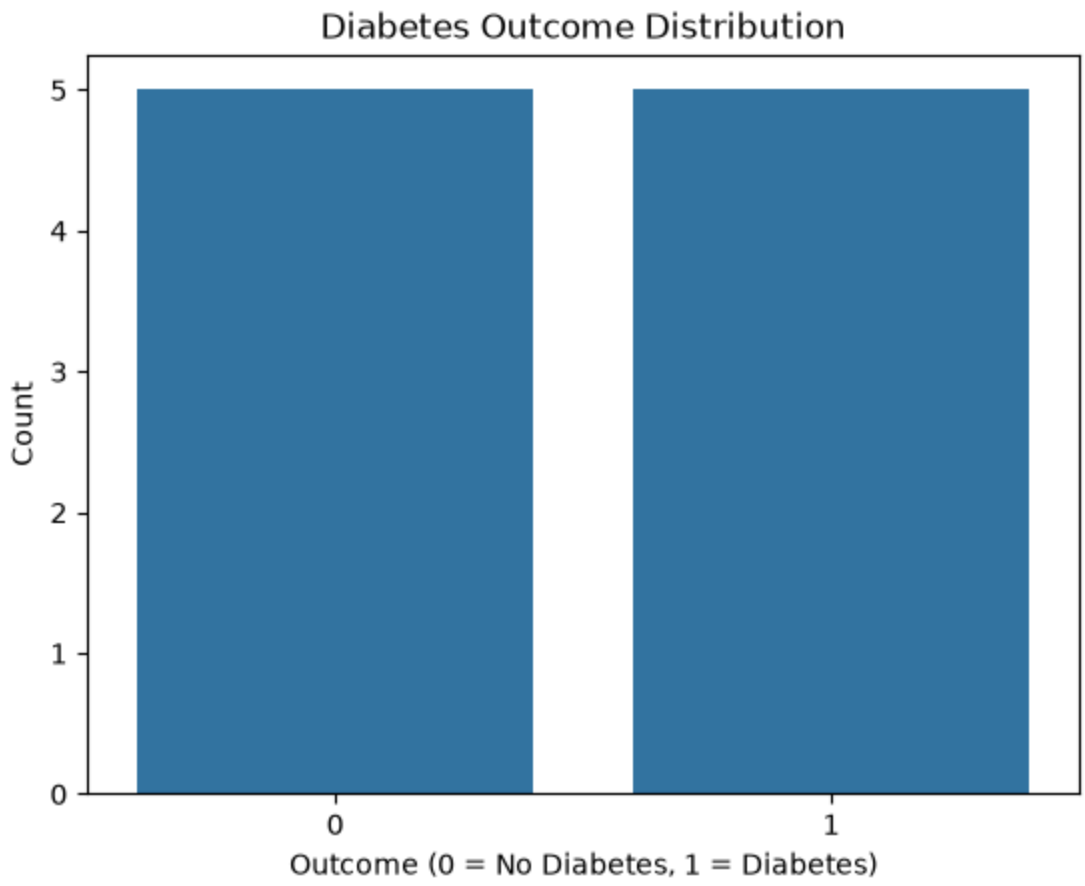
```
Missing Values:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64

Statistical Summary:
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count    10.000000   10.000000      10.000000      10.000000   10.000000
mean      4.000000  125.800000      59.400000      19.900000   89.300000
std       3.265986   40.449694      25.051946      17.978073  169.937276
min       0.000000   78.000000       0.000000       0.000000    0.000000
25%       1.250000   94.250000      53.500000       0.000000    0.000000
50%       3.500000  115.500000      66.000000      26.000000    0.000000
75%       5.750000  145.250000      71.500000      34.250000   92.500000
max      10.000000  197.000000      92.000000      45.000000  543.000000

             BMI  DiabetesPedigreeFunction        Age    Outcome
count  10.000000                  10.00000  10.000000  10.000000
mean   31.470000                   0.50370  33.500000   0.500000
std     6.050537                   0.65616  10.102255   0.527046
min    23.300000                   0.13400  21.000000   0.000000
25%    26.975000                   0.17300  29.250000   0.000000
50%    30.750000                   0.22450  30.500000   0.500000
75%    34.875000                   0.55800  32.750000   1.000000
max    43.100000                   2.28800  53.000000   1.000000

Number of Duplicate Rows:
0
```


Correlation Heatmap of Diabetes Dataset
Pregnancies 1.00 0.25 -0.37 -0.66 -0.43 -0.17 -0.33 0.04 -0.06
Glucose 0.25 1.00 0.11 0.12 0.53 -0.04 0.21 0.74 0.59
BloodPressure -0.37 0.11 1.00 0.07 0.06 -0.30 -0.20 0.22 -0.01
SkinThickness -0.66 0.12 0.07 1.00 0.64 0.21 0.29 0.51 0.56
Insulin -0.43 0.53 0.06 0.64 1.00 0.12 0.06 0.56 0.44
BMI -0.17 -0.04 -0.30 0.21 0.12 1.00 0.57 0.11 0.14
DiabetesPedigreeFunction -0.33 0.21 -0.20 0.29 0.06 0.57 1.00 0.08 0.47
Age 0.04 0.74 0.22 0.51 0.56 0.11 0.08 1.00 0.55
Outcome -0.06 0.59 -0.01 0.56 0.44 0.14 0.47 0.55 1.00
Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6


Diabetes Outcome Distribution
Count
0
1
2
3
4
5
0
1
Outcome (0 = No Diabetes, 1 = Diabetes)

In [ ]:

Name: Thamizhselvan.R Register Number: 250701780

In [1]:
```
import pandas as pd
```

```
# Create sample website traffic data
np.random.seed(42)

dates = pd.date_range(start='2025-01-01', periods=120, freq='D')
page_views = (
    500
    + np.arange(120) * 2
    + 50 * np.sin(2 * np.pi * np.arange(120) / 7)
    + np.random.normal(0, 20, 120)
)

# Add a few traffic spikes
page_views[40] += 200
page_views[85] += 250

data = pd.DataFrame({
    'Date': dates,
    'Page_Views': page_views
})

data.set_index('Date', inplace=True)

# Display the dataset
print("Website Traffic Data:")
print(data.head())

# 1. Time Series Decomposition
decomposition = seasonal_decompose(
    data['Page_Views'],
    model='additive',
    period=7
)

decomposition.plot()
plt.suptitle("Time Series Decomposition", y=1.02)
plt.tight_layout()
plt.show()

# 2. Moving Average
data['Page_Views_MA'] = data['Page_Views'].rolling(
    window=7
).mean()

data[['Page_Views', 'Page_Views_MA']].plot(
    figsize=(10, 5)
)
plt.title("Page Views with Moving Average")
plt.xlabel("Date")
plt.ylabel("Page Views")
```

```
plt.show()

# 3. Anomaly Detection using Z-score
mean = data['Page_Views'].mean()
std = data['Page_Views'].std()

data['Page_Views_Z'] = (
    data['Page_Views'] - mean
) / std

anomalies = data[data['Page_Views_Z'].abs() > 3]

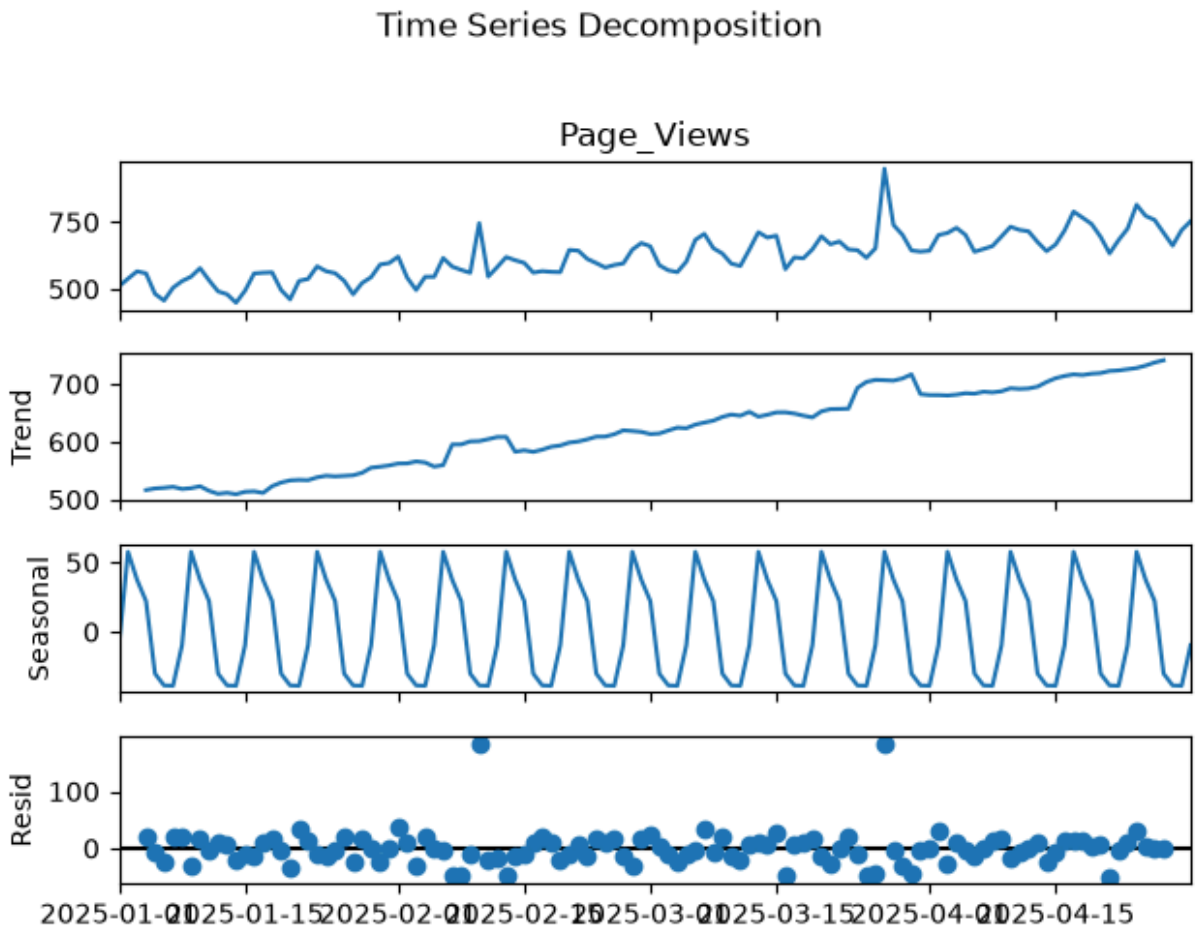
print("\nDetected Anomalies:")
print(anomalies[['Page_Views', 'Page_Views_Z']])

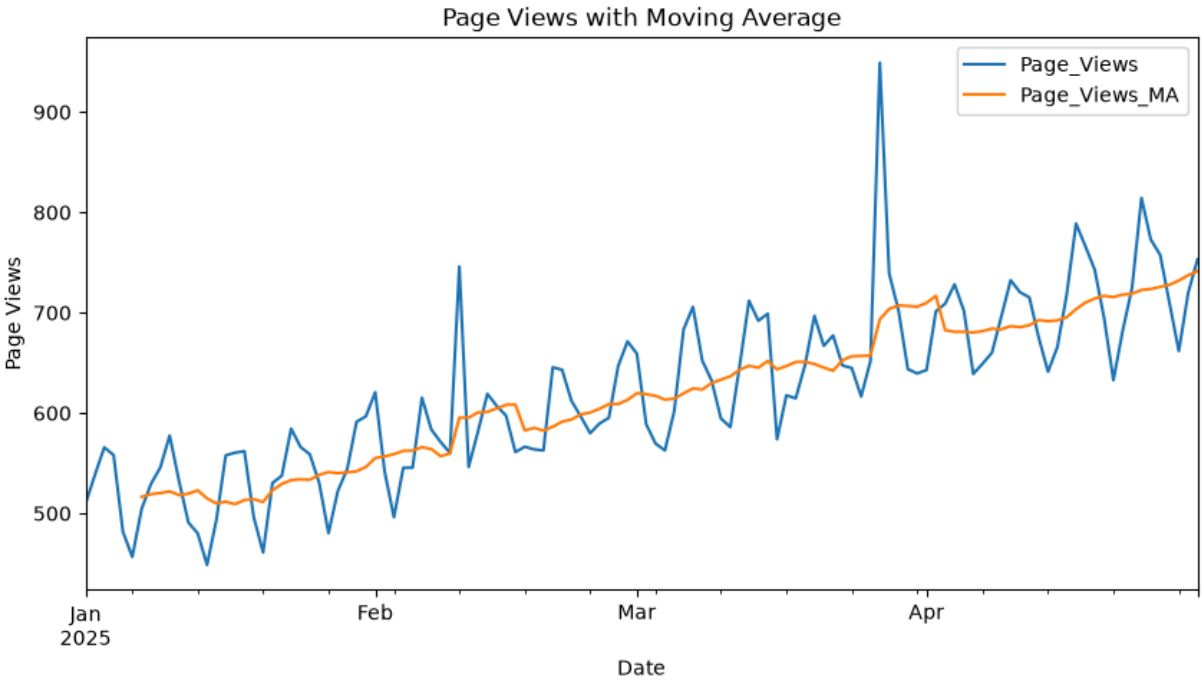
# Plot anomalies
plt.figure(figsize=(10, 5))
plt.plot(
    data.index,
    data['Page_Views'],
    label='Page Views'
)

plt.scatter(
    anomalies.index,
    anomalies['Page_Views'],
    color='red',
    label='Anomalies'
)

plt.title("Website Traffic Anomaly Detection")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.legend()
plt.show()
```

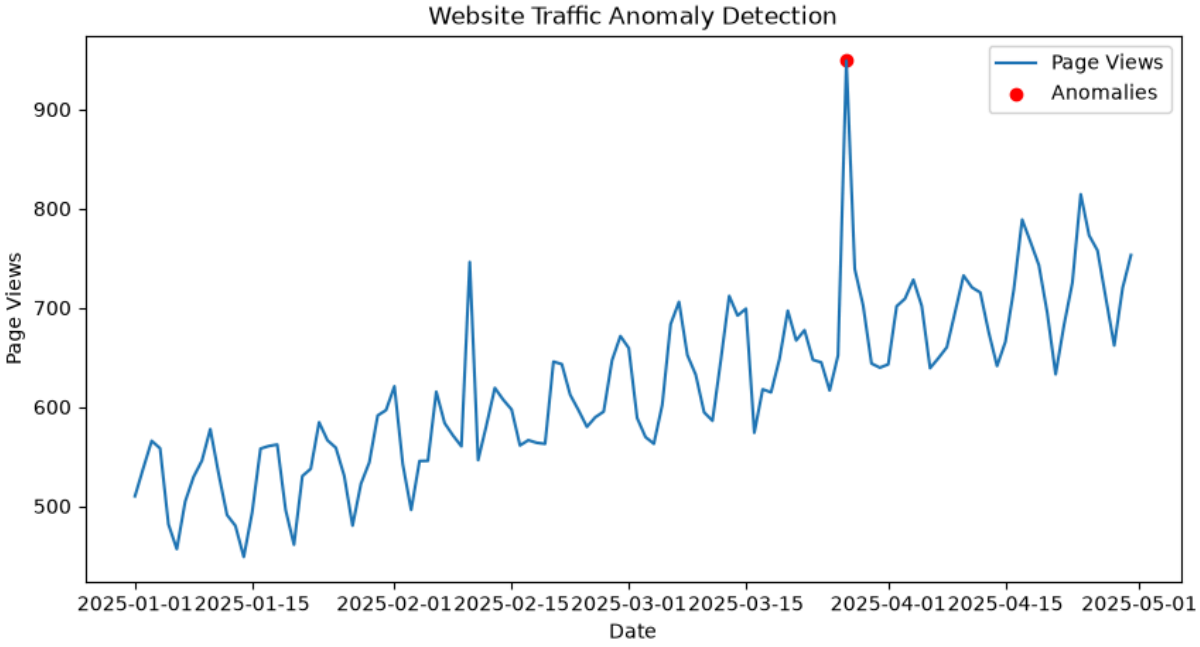
```
Website Traffic Data:
            Page_Views
Date
2025-01-01  509.934283
2025-01-02  538.326288
2025-01-03  565.700166
2025-01-04  558.154784
2025-01-05  481.622746
```





Detected Anomalies:

Date	Page_Views	Page_Views_Z
2025-03-27	949.056433	3.823627


Website Traffic Anomaly Detection
Page Views
Anomalies
900
800
700
600
500
Page Views
2025-01-01
2025-01-15
2025-02-01
2025-02-15
2025-03-01
2025-03-15
2025-04-01
2025-04-15
2025-05-01
Date

In [ ]:

Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Random Sampling and Sampling Distribution

```
import numpy as np
import matplotlib.pyplot as plt

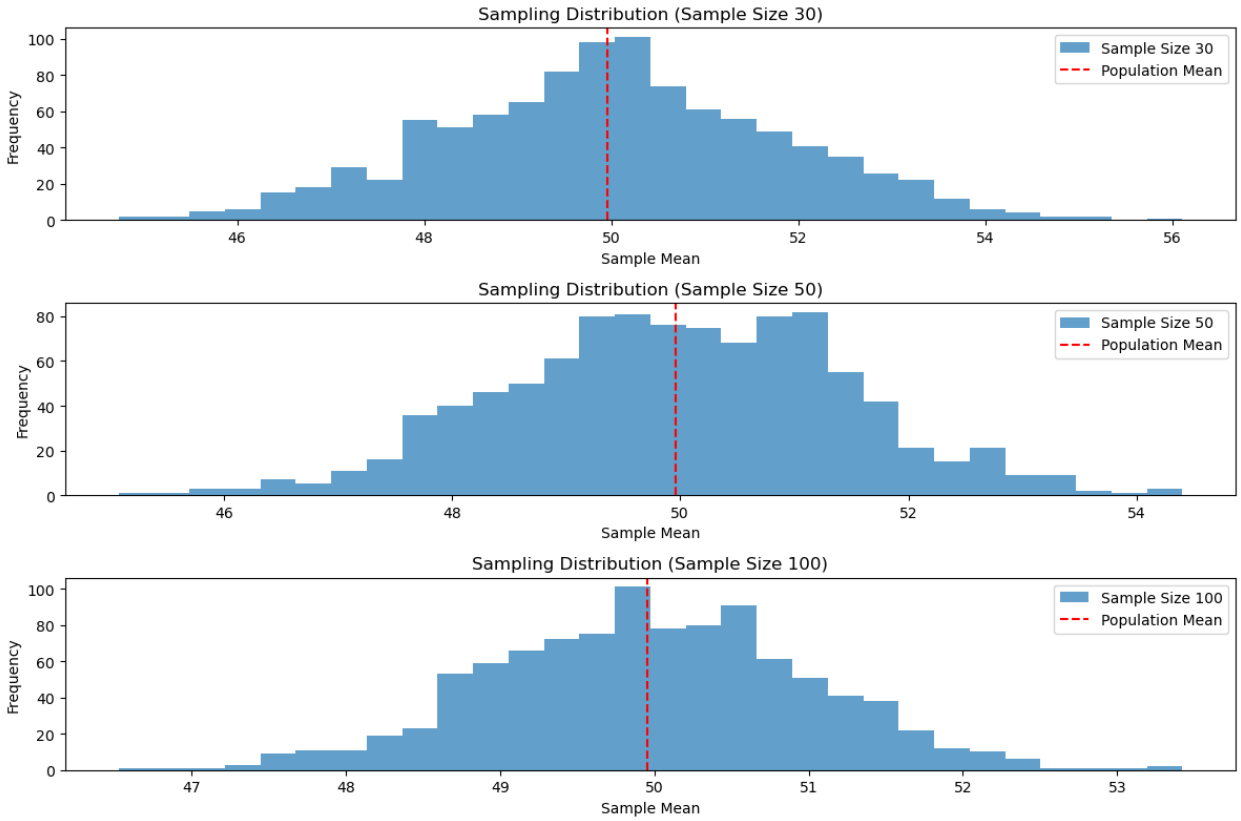
# Step 1: Generate a population (e.g., normal distribution)
population_mean = 50
population_std = 10
population_size = 100000
population = np.random.normal(population_mean, population_std,
population_size)

# Step 2: Random sampling
sample_sizes = [30, 50, 100] # different sample sizes to consider
num_samples = 1000 # number of samples for each sample size
sample_means = {}

for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size,
replace=False)
        sample_means[size].append(np.mean(sample))

# Step 3: Plotting sampling distributions
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i+1)
    plt.hist(sample_means[size], bins=30, alpha=0.7, label=f'Sample
Size {size}')
    plt.axvline(np.mean(population), color='red', linestyle='dashed',
linewidth=1.5, label='Population Mean')
    plt.title(f'Sampling Distribution (Sample Size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()
plt.tight_layout()
plt.show()
```


Sampling Distribution (Sample Size 30)
Frequency
Sample Mean
Sample Size 30
Population Mean
Sampling Distribution (Sample Size 50)
Frequency
Sample Mean
Sample Size 50
Population Mean
Sampling Distribution (Sample Size 100)
Frequency
Sample Mean
Sample Size 100
Population Mean

Name: Thamizhselvan.V

Register Number: 250701780

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

In [3]:
```
data=pd.read_csv('Iris.csv')
data
```

Out[3]:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Virginica
146	6.3	2.5	5.0	1.9	Virginica
147	6.5	3.0	5.2	2.0	Virginica
148	6.2	3.4	5.4	2.3	Virginica
149	5.9	3.0	5.1	1.8	Virginica

150 rows × 5 columns

In [4]:
```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    str    
dtypes: float64(4), str(1)
memory usage: 7.2 KB
```

In [5]:
```
data.describe()
```

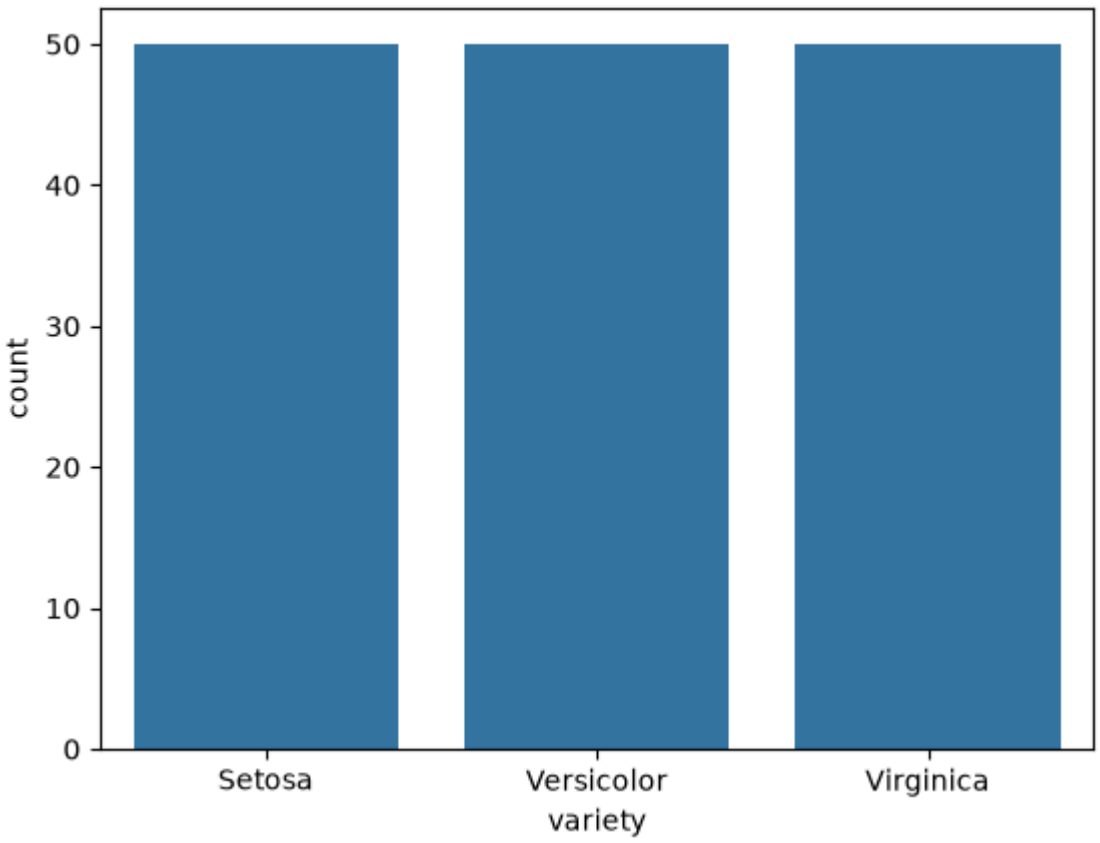
Out[5]:

	sepal.length	sepal.width	petal.length	petal.width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

In [6]:
```
data.value_counts('variety')
```

Out[6]:
```
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

In [7]:
```
sns.countplot(x='variety',data=data,)
plt.show()
```



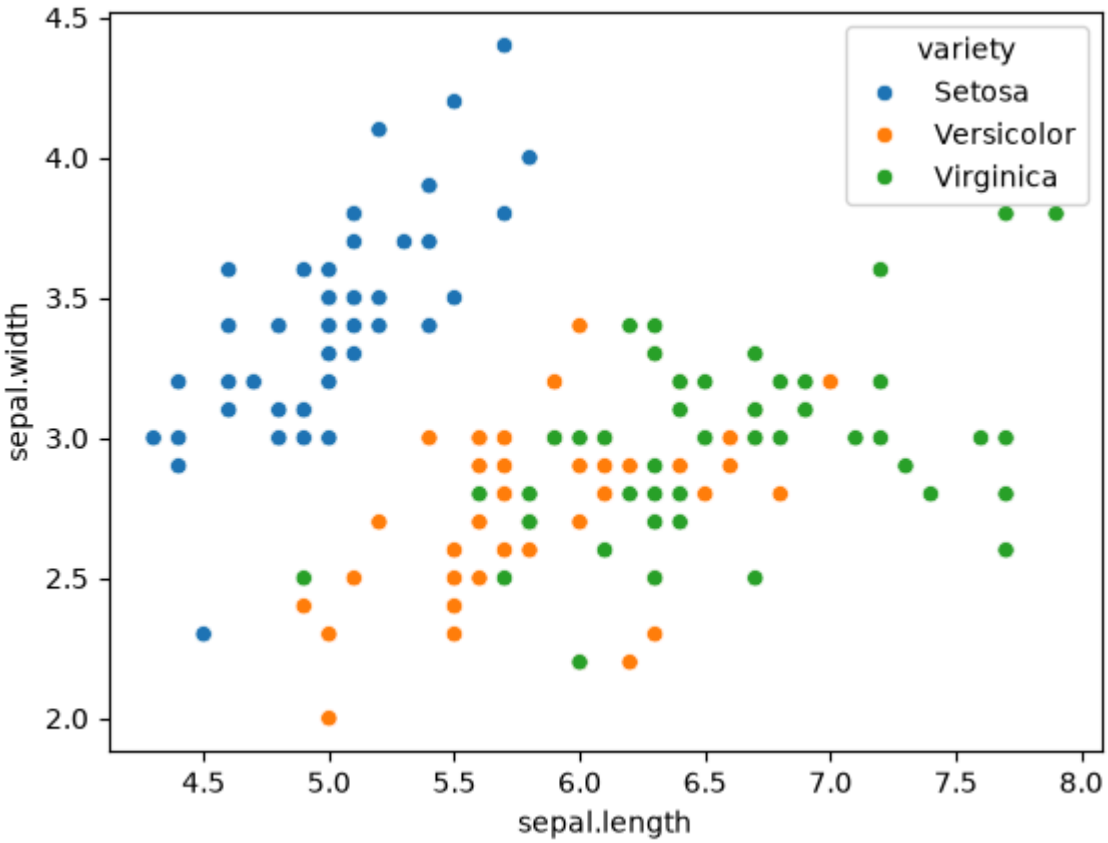
In [8]:
```
dummies=pd.get_dummies(data.variety)
FinalDataset=pd.concat([pd.get_dummies(data.variety),data.iloc[:,[0,1,2,3]]],axi
FinalDataset.head()
```

Out[8]:

	Setosa	Versicolor	Virginica	sepal.length	sepal.width	petal.length	petal.width
0	True	False	False	5.1	3.5	1.4	0.2
1	True	False	False	4.9	3.0	1.4	0.2
2	True	False	False	4.7	3.2	1.3	0.2
3	True	False	False	4.6	3.1	1.5	0.2
4	True	False	False	5.0	3.6	1.4	0.2

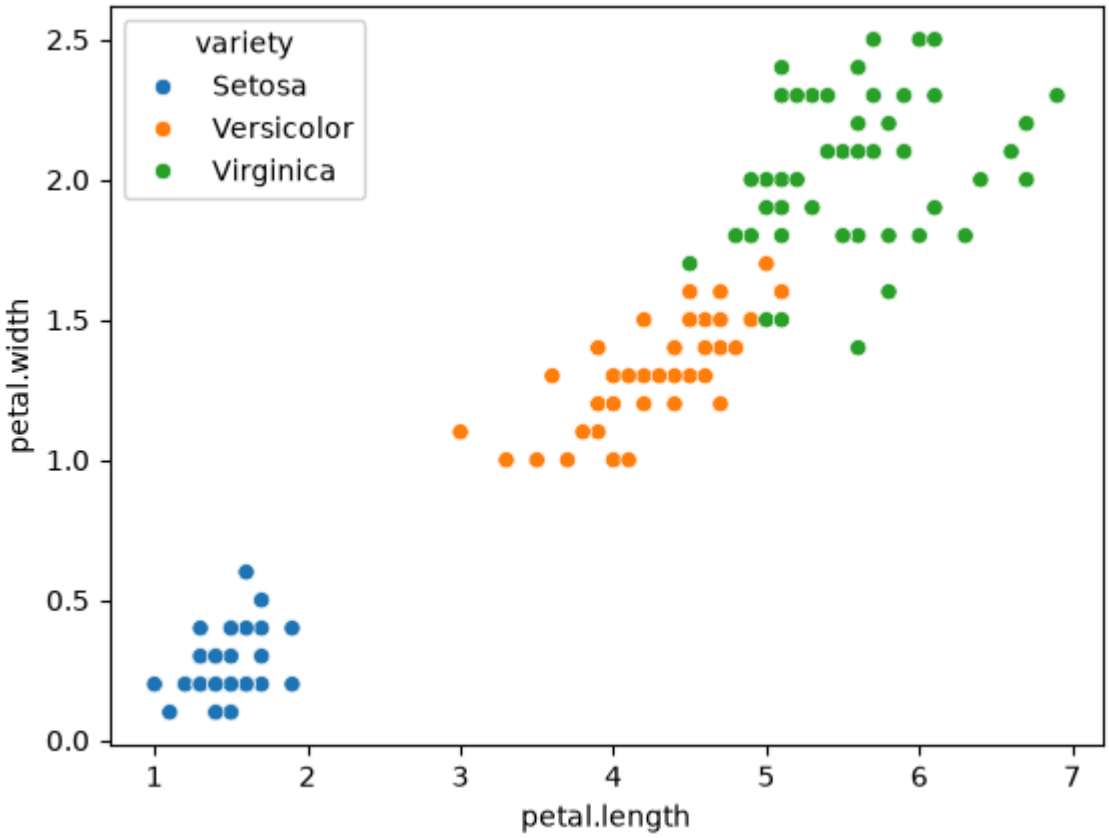
In [11]:
```
sns.scatterplot(x='sepal.length',y='sepal.width',hue='variety',data=data)
```

Out[11]:
```
<Axes: xlabel='sepal.length', ylabel='sepal.width'>
```

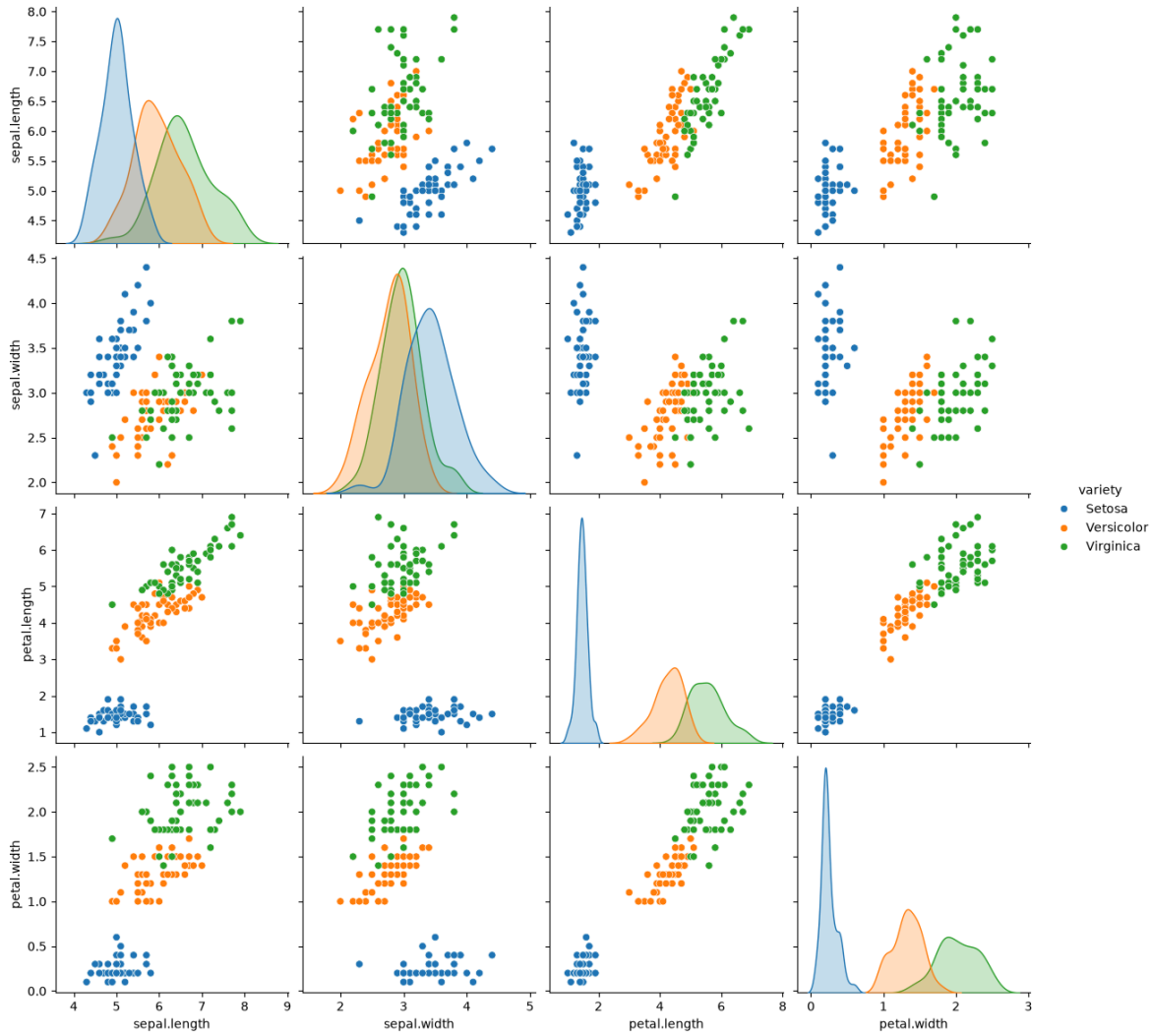


In [13]:
```
sns.scatterplot(x='petal.length',y='petal.width',hue='variety',data=data)
```

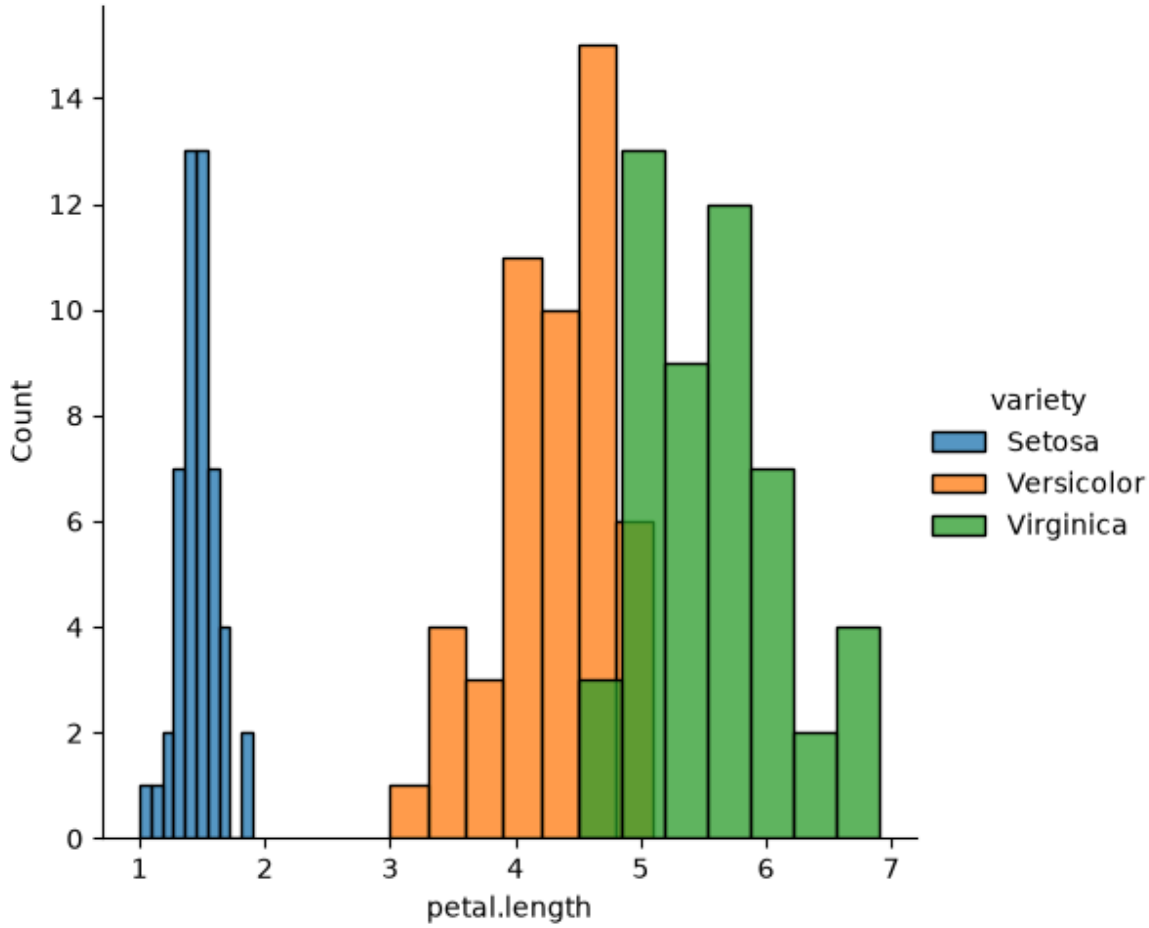
Out[13]:
```
<Axes: xlabel='petal.length', ylabel='petal.width'>
```



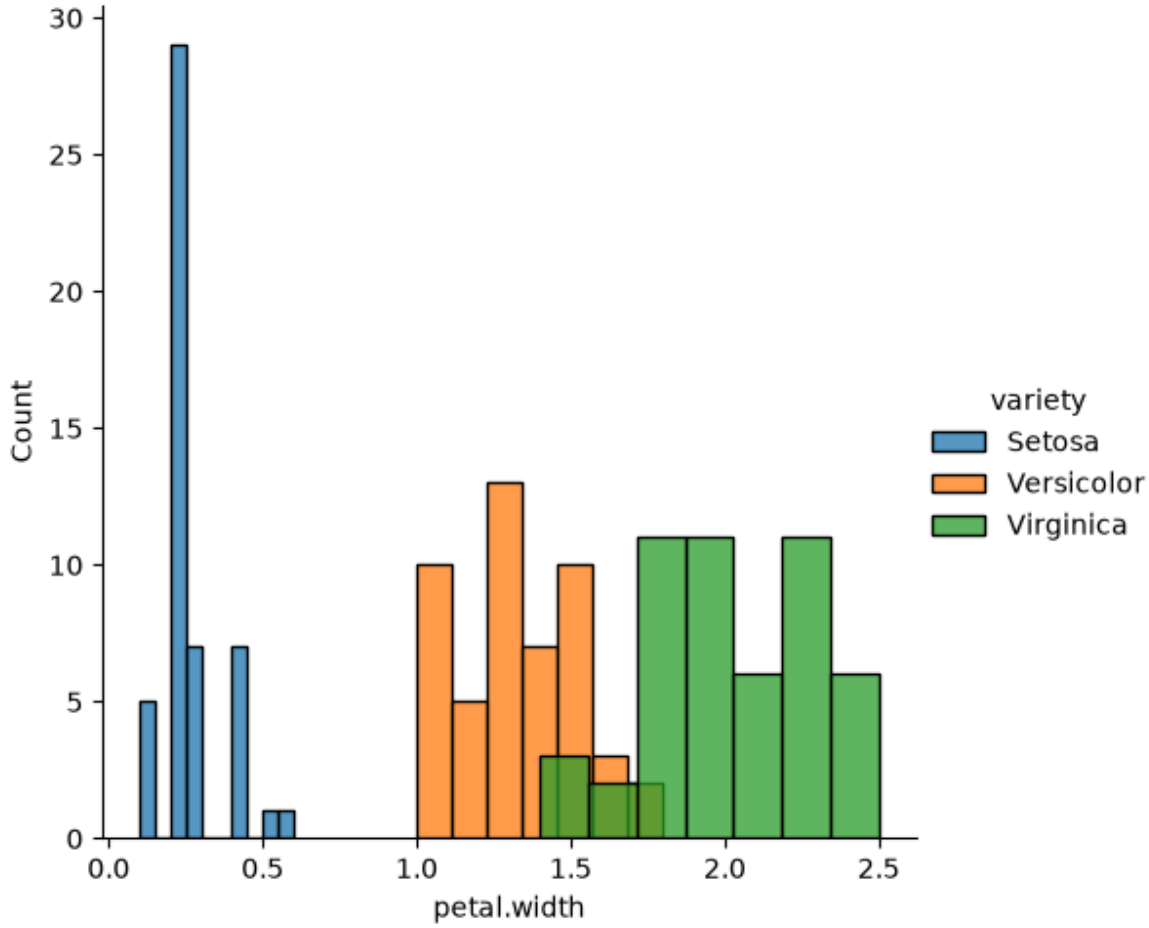
In [14]:
```
sns.pairplot(data,hue='variety',height=3);
```



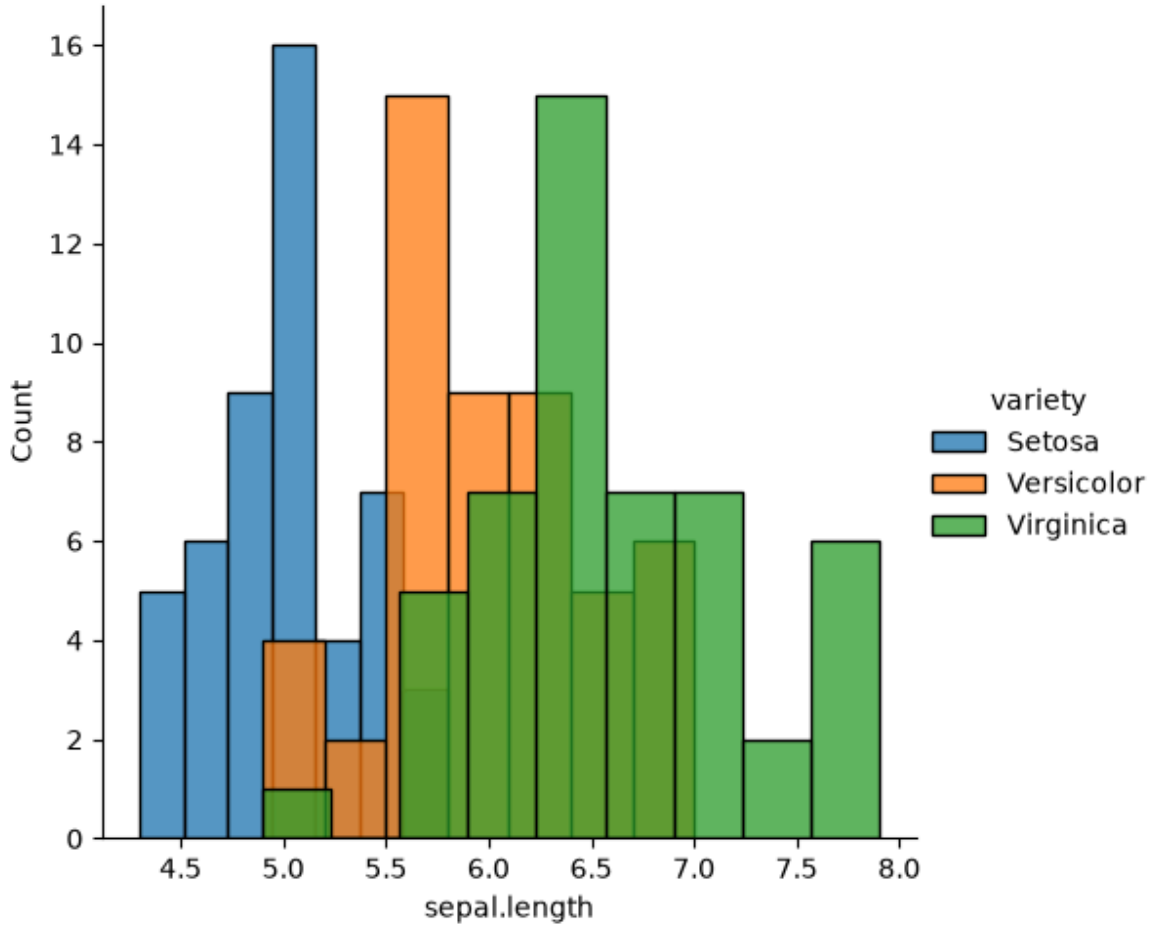
In [17]:
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'petal.length').add_
```



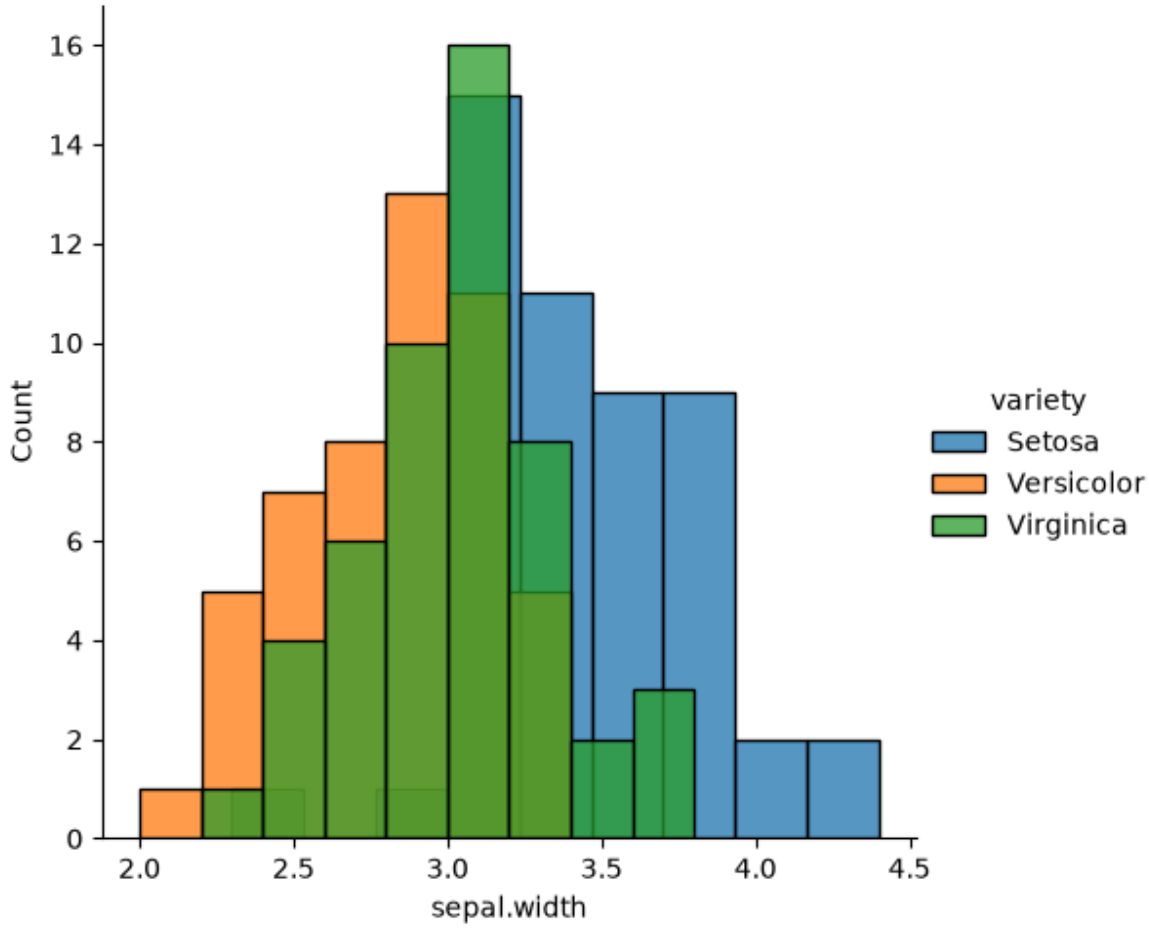
In [19]:
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'petal.width').add_l
plt.show();
```



In [20]:
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'sepal.length').add_
plt.show();
```



In [21]:
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'sepal.width').add_l
plt.show();
```


16
14
12
10
8
6
4
2
0
Count
2.0
2.5
3.0
3.5
4.0
4.5
sepal.width
variety
Setosa
Versicolor
Virginica

In [ ]:

Name: Thamizhselvan.VRegister Number: 250701780

Experiment Name: Array Functions in Data Science

```
import numpy as np
array=np.random.randint(1,100,9)
array
```

```
array([ 6, 11, 49, 88, 98, 80, 24, 43, 12], dtype=int32)
```

```
np.sqrt(array)
```

```
array([2.44948974, 3.31662479, 7.        , 9.38083152, 9.89949494,
       8.94427191, 4.89897949, 6.55743852, 3.46410162])
```

```
array.ndim
```

```
1
```

```
new_array=array.reshape(3,3)
```

```
new_array
```

```
array([[ 6, 11, 49],
       [88, 98, 80],
       [24, 43, 12]], dtype=int32)
```

```
new_array.ndim
```

```
2
```

```
new_array.ravel()
```

```
array([ 6, 11, 49, 88, 98, 80, 24, 43, 12], dtype=int32)
```

```
newm=new_array.reshape(3,3)
```

```
newm
```

```
array([[ 6, 11, 49],
       [88, 98, 80],
       [24, 43, 12]], dtype=int32)
```

```
newm[2,1:3]
```

```
array([43, 12], dtype=int32)
```

```
newm[1:2,1:3]
```

```
array([[98, 80]], dtype=int32)
```

```
new_array[0:3,0:0]
```

```
array([], shape=(3, 0), dtype=int32)
```

```
new_array[0:2,0:1]

array([[ 6],
       [88]], dtype=int32)

new_array[0:3,0:1]

array([[ 6],
       [88],
       [24]], dtype=int32)

new_array[1:3]

array([[88, 98, 80],
       [24, 43, 12]], dtype=int32)
```

Name:Thamizhselvan.R Register Number:250701780

In [1]:
```
import numpy as np
import pandas as pd

# 1. Create a DataFrame from a list
data = [
    [1, 'Smith', 50000],
    [2, 'Jones', 60000]
]

df = pd.DataFrame(data)
df.columns = ['Empd', 'Name', 'Salary']

print("Employee DataFrame:")
print(df)

# 2. Display DataFrame information
print("\nDataFrame Information:")
df.info()

# 3. Create an employee dataset
employee_data = {
    'emp id': [1, 2, 3, 4, 5, 6, 7],
    'name': [
        'SREE VARSSINI K S',
        'SREEMATHI B',
        'SREYA G',
        'SREYASKARI MULLAPUDI',
        'SRI AKASH U G',
        'SRI HARSHAVARDHANAN R',
        'SRI HARSHAVARDHANAN R'
    ],
    'salary': [5000, 6000, 7000, 5000, 8000, 3000, 6000]
}

df = pd.DataFrame(employee_data)

# 4. Display the first and last five rows
print("\nFirst Five Rows:")
print(df.head())

print("\nLast Five Rows:")
print(df.tail())

# 5. Display DataFrame information
print("\nEmployee DataFrame Information:")
df.info()

# 6. Select the salary column
print("\nSalary Column:")
print(df['salary'])

# 7. Calculate statistical measures
print("\nMean Salary:", df['salary'].mean())
print("Median Salary:", df['salary'].median())
```

```python
print("Mode Salary:", df['salary'].mode().tolist())
print("Salary Variance:", df['salary'].var())
print("Salary Standard Deviation:", df['salary'].std())

# 8. Display numerical summary
print("\nStatistical Summary:")
print(df.describe())

# 9. Display summary of all columns
print("\nComplete Summary:")
print(df.describe(include='all'))

# 10. Display column names
print("\nColumn Names:")
print(df.columns)

# 11. Convert DataFrame into a NumPy array
emp_array = df.values
print("\nEmployee Array:")
print(emp_array)

# 12. Create a DataFrame from the NumPy array
employee_DF = pd.DataFrame(emp_array, columns=df.columns)

print("\nFinal Employee DataFrame:")
print(employee_DF)
```

```
Employee DataFrame:
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000

DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      str
 2   Salary  2 non-null      int64
dtypes: int64(2), str(1)
memory usage: 190.0 bytes

First Five Rows:
   emp id                  name  salary
0       1     SREE VARSSINI K S    5000
1       2           SREEMATHI B    6000
2       3               SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH U G    8000

Last Five Rows:
   emp id                   name  salary
2       3                SREYA G    7000
3       4   SREYASKARI MULLAPUDI    5000
4       5          SRI AKASH U G    8000
5       6  SRI HARSHAVARDHANAN R    3000
6       7  SRI HARSHAVARDHANAN R    6000

Employee DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  7 non-null      int64
 1   name    7 non-null      str
 2   salary  7 non-null      int64
dtypes: int64(2), str(1)
memory usage: 410.0 bytes

Salary Column:
0    5000
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: salary, dtype: int64

Mean Salary: 5714.285714285715
```

```
Median Salary: 6000.0
Mode Salary: [5000, 6000]
Salary Variance: 2571428.5714285714
Salary Standard Deviation: 1603.5674514745463

Statistical Summary:
         emp id       salary
count  7.000000     7.000000
mean   4.000000  5714.285714
std    2.160247  1603.567451
min    1.000000  3000.000000
25%    2.500000  5000.000000
50%    4.000000  6000.000000
75%    5.500000  6500.000000
max    7.000000  8000.000000

Complete Summary:
          emp id                    name       salary
count   7.000000                       7     7.000000
unique       NaN                       6          NaN
top          NaN  SRI HARSHAVARDHANAN R          NaN
freq         NaN                       2          NaN
mean    4.000000                     NaN  5714.285714
std     2.160247                     NaN  1603.567451
min     1.000000                     NaN  3000.000000
25%     2.500000                     NaN  5000.000000
50%     4.000000                     NaN  6000.000000
75%     5.500000                     NaN  6500.000000
max     7.000000                     NaN  8000.000000

Column Names:
Index(['emp id', 'name', 'salary'], dtype='str')

Employee Array:
[[1 'SREE VARSSINI K S' 5000]
 [2 'SREEMATHI B' 6000]
 [3 'SREYA G' 7000]
 [4 'SREYASKARI MULLAPUDI' 5000]
 [5 'SRI AKASH U G' 8000]
 [6 'SRI HARSHAVARDHANAN R' 3000]
 [7 'SRI HARSHAVARDHANAN R' 6000]]

Final Employee DataFrame:
  emp id                   name salary
0      1      SREE VARSSINI K S   5000
1      2            SREEMATHI B   6000
2      3                SREYA G   7000
3      4   SREYASKARI MULLAPUDI   5000
4      5          SRI AKASH U G   8000
5      6  SRI HARSHAVARDHANAN R   3000
6      7  SRI HARSHAVARDHANAN R   6000
```

In [ ]: